A NEW TECHNIQUE FOR DISTRIBUTED NABTICS DATA REPLICATION MANAGEMENT

EMMA AHMAD SIRAJUDIN

Thesis submitted in Fulfillment for the Requirement for the Degree of Masters of Science in the Faculty of Informatics and Applied Mathematics

Universiti Malaysia Terengganu

MAY 2015

To Shapiah Zainal Abidin and in memory of Ahmad Sirajudin Abu Bakar,

my first and foremost teachers.

Abstract of thesis presented to the Senate of Universiti Malaysia Terengganu in fullfillment of the requirement for the degree of Master of Science

A NEW TECHNIQUE FOR DISTRIBUTED DATA REPLICATION MANAGEMENT

EMMA AHMAD SIRAJUDIN

2015

Faculty: Informatics and Applied Mathematics

Adding availability to a system is similar to adding more reliability where an available system is guaranteed to be reliable while a reliable system may not necessarily be available. Availability, being the probability a system is functional at a time is a feature that is sought after and deemed obligatory in some system. The knowledge repository for National Bioinformatics System or NABTICS is used by researchers on regular basis as the research in microbes and algae progresses. Such a system needs availability to bring continuity to research tasks as well as to keep data integrity. An autonomous approach is desired to ease the tasks and ensure correctness as well as to let the user experience virtually zero downtime by executing a prompt recovery. To realize this a fault tolerant framework is required for the system. The first requirement for the framework is redundancy of components in order to reduce single points of failure (SPOF) by adding more parallelness in the system. This is delivered by disseminating the system into separate modules that are independent of each

other and maintaining replication of neighboring modules in each module server. Next a mechanism to check real-time availability of each module is implemented so that a fault can be detected with promptness and accuracy. In effect of detecting a fault, a recovery action is initiated to request IP take-over in one of the neighboring sites essentially activating the replicated module. As a result of these techniques, the overall NABTICS system availability has improved by 70% and furthermore the eventuality of a failed server has been tested and user does not experience a significant downtime and the failure does not result in service interruption. In other word, availability is restored at an event of site failure. The NABTICS is said to have higher availability score compared to its original implementation.

Abstrak tesis yang dikemukan kepada Senat Universiti Malaysia Terengganu sebagai memenuhi keperluan untuk Ijazah Master Sains

TEKNIK BARU UNTUK PENGURUSAN REPLIKASI DATA TERAGIH

EMMA AHMAD SIRAJUDIN

2015

Fakulti: Informatik dan Matematik Gunaan

Menyediakan kebolehsediaan kepada sesuatu sistem sama seperti menyediakan kebolehharapan di mana satu sistem yang terbolehsedia sudah pasti merupakan satu sistem yang terbolehharap sebaliknya sistem yang terbolehharap tidak semestinya satu sistem yang terbolehsedia. Kebolehsediaan, yang bermaksud kebarangkalian sesuatu sistem berfungsi pada satu-satu masa adalah satu ciri yang diinginkan dan dianggap wajib dalam sesetengah sistem. Repositori pengetahuan Sistem Bioinformatik Kebangsaan atau NABTICS digunakan oleh penyelidik secara tetap sejajar dengan kemajuan penyelidikan mikrob dan alga. Sitem seumpamanya memerlukan kebolehsediaan bagi memastikan tugasan penyelidikan adalah selanjar dan bagi memastikan integriti data. Pendekatan berautonomi diinginkan bagi memudahkan tugas dan memastikan ketepatan selanjutnya dengan pengesanan kerosakan yang pantas pengguna tidak merasakan sebarang masa rosak. Untuk mewujudkan keadaan ini, satu rangkakerja yang toleran kepada kerosakan diperlukan. Keperluan yang

pertama untuk rangkakerja ini adalah pengulangan komponen untuk menghapuskan kerosakan poin tunggal (SPOF) dengan menambahkan keselarian di dalam sistem. Ini dihasilkan dengan mengagihkan modul-modul sistem yang tidak bersandar kepada satu sama lain dan mengekalkan salinan modul-modul bersebelahan dalam setiap server modul. Seterusnya satu mekanisma untuk mengawasi kebolehsediaan setiap modul pada masa nyata dibina supaya kerosakan dapat dikesan dengan pantas dan tepat. Secara efek dengan mengesan kerosakan, pemulihan dimulakan dengan mengarahkan pengambil-alihan IP oleh salah satu server bersebelahan dan dengan itu mengaktifkan salinan modul. Hasilnya kebolehsediaan total sistem NABTICS telah ditingkatkan sebanyak 70%. Kepastian sesuatu kerosakan server telah diuji di mana pengguna tidak merasakan masa rosak yang signifikan dan tiada gangguan servis. Boleh dikatakan sistem NABTICS mempunyai skor kebolehsediaan yang lebih tinggi berbanding yang asal.

ACKNOWLEDGEMENTS

First and foremost, I am really grateful for this opportunity and this blessing from the Almighty.

I would like to record my appreciation to Prof. Md. Yazid Mohd Saman, Dr. Ahmad Shukri Mohd Noor, Dr. Mustafa Man and Mr. Mohamad Nor Hassan for introducing me to many topics and for their support. Also to a number of fellow graduate students for their sharing and contribution and for keeping me motivated. I feel very fortunate to be their student and acquaintance.

Special thanks to my school teachers for inspiring me early on and for making me enjoy learning and exploring.

Last but not least, I would like to thank my support system; my family, my husband and my little son for everything that they have done and given me especially in helping me cope during the course of my thesis preparation.

May our gathering eventually be in Jannah.

CONTENTS

1 Introduction **1**

1.1 Research Background 1

1.2 Problem Statement 4

1.3 Objectives 5

1.4 Research Scope 6

1.5 Contributions 6

1.6 Thesis Organization 7

2 Literature Review **10**

2.1 Introduction 10

2.2 Availability and Unavailability 10

2.3 Mean Time Between Failures (MTBF) 12

2.4 Mean Time To Failure (MTTF) 13

2.5 Mean Time To Repair (MTTR) 13

2.6 Probability of availability 14

2.7 Failure Rates 14

2.8 System availability 16

2.8.1 Availability in Series 16

2.8.2 Availability in parallel 17

2.8.3 Availability in joint parallel and series 18

2.8.4 Availability in distributed system 19

2.9 Terminology 20

2.10 Behavior of Failed System 24

2.11 Failure Detection 25

2.11.1 Properties 25

2.11.2 Quality of Service (QoS) 26

2.12 Approaches to Failure Detection 27

2.12.1 Affirmative Adaptive Failure Detection (AAFD) . . . 29

2.12.2 Gossip-style Failure Detection 31

2.13 Load-balancing and Fail-over Clustering: A Comparison . . . 32

2.14 Failure Recovery 33

2.15 Failure Recovery using Replication Techniques 34

2.15.1 Two Replica Distribution Technique (TRDT) 35

2.15.2 Neighbor Replica Distribution Technique (NRDT) . . 36

2.16 Resource Management System for High Availability 38

2.17 Summary 40

3 Methodology **43**

3.1 Introduction 43

3.2 Server Cluster 46

3.3 Neighbor replication 48

3.4 MySQL Database Replication 49

3.5 High Availability Framework 53

3.5.1 The C Socket Heartbeat Transaction 55

3.5.2 Confirming Failure 63

3.5.3 Selecting for Recovery 64

3.6 Conclusion 66

4 Implementation **69**

4.1 Introduction 69

4.2 Hardware Specification 70

4.3 Software Specification 72

4.4 Development 75

4.5 Server Cluster 75

4.6 Setting up for running 76

4.7 Setting up proxy 77

4.8 NABTICS System 78

4.9 Setting up MySQL for Database Replication 80

4.10 Setting up SSH for secure remote command 83

4.11 Fault Detection Configuration 85

4.12 Failure Recovery Configuration 87

4.13 Executing the fault tolerant framework 88

4.14 Recovery process 89

4.15 Conclusion 91

5 Results and Analysis **92**

5.1 Introduction 92

5.2 Failure detection analysis 92

5.3 Failure recovery analysis 99

5.4 System Performance 101

5.4.1 The serial and parallel model of the system 102

5.4.2 From Failure to Recovery 105

5.5 Conclusion 111

6 Conclusions **112**

6.1 Conclusion 112

6.2 Future Work 114

A Index Logs **136**

B Source Codes **138**

LIST OF TABLES

3.1 The neighbor relationship of the nodes and the modules they contain 49

3.2 Algorithm for Heartbeat Monitor 61

4.1 Server types and network settings 71

4.2 Host table 76

A.1 Neighbors relationship among servers 137

A.2 Node's Information 137

A.3 Node's Weight to Determine Best Neighbor 137

LIST OF FIGURES

2.1 Availability in series 17

2.2 Availability in parallel 18

2.3 Availability in joint parallel and series 18

2.4 TRDT data replica distribution technique when $N = 2n$ 36

2.5 NRDT with 3 x 3 grid 37

2.6 NRDT with 5 nodes 38

3.1 The Methodology Phases 44

3.2 System Overview 45

3.3 Database arrangement using MySQL Replication 52

3.4 The portal operations at Heartbeat Monitor 56

3.5 The C Socket for Heartbeat Transaction 58

3.6 Flowchart of Fault Detection Process 59

3.7 Flowchart of Failure Recovery Process 65

4.1 NABTICS modules setting in Proxy server 78

4.2 NABTICS Homepage's Links 79

4.3 The fault-tolerant framework 88

5.1 Heartbeat sending and receiving 93

5.2 Heartbeat sending and receiving for longer sample 94

5.3 Distribution of heartbeat generation time 95

5.4 Distribution of heartbeat receiving time 95

5.5 Expected time of arrival adapting to heartbeat inter-arrival time 97

5.6 Joint serial and paralel model in distributed NABTICS 102

5.7 Monitoring heartbeat from node4 106

5.8 A timeout occured but is not a node failure 107

5.9 Failure is detected and confirmed 108

5.10 Before node2 becoming a fail-over 109

5.11 Node2 becoming a fail-over 109

Chapter 1

INTRODUCTION

An introduction to the research is given in this chapter in order to describe the problem and cover the underlying background to the research project. The objective of this research is outlined to be elaborated further in the rest of the chapters. The structure of the thesis is also specified.

1.1 Research Background

The ability of a system to be functional at any given instance is the measure of its availability while its ability to be functional over a time period is the measure of its reliability [1]. In some systems, high availability is a critical attribute where being temporarily out of service could spell disaster or lost of lives. Such systems are mission critical for example are military, aviation or telecommunication systems. Some trend users demand the availability of 99.99% which is equal to only one hour of unavailability in a year [2]. Such system with near perfect availability is difficult to achieve. This is because eliminating a single point of failure (SPOF) requires complex design and robustness in hardware and software.

Ideally, replication should monotonically increase availability [3]. The concept of having redundancy in parallel effectively reduces the cost of SPOF by spreading it onto multiple sites thereby ensuring the system is alive as long as there is at least one site online. Having more computers working together as such in a distributed system, decreases the possibility that a single failure can disrupt the entire system. Renesse et al. [4] stated that one of the benefits of distributed system is the increase of parallelism for replication. Parallel systems optimize its availability by reducing unavailability with its very nature. The current approaches in this area tend to focus on increasing parallelness in the system to decrease unavailability. By masking errors in the replicated components, high availability is achieved in distributed environments through replication [5].

Distributed systems can easily be made redundant, which is the key to many fault-tolerance techniques. Unfortunately, distribution also means that the imperfect and fault-prone physical world cannot be ignored, so that as much as they help in supporting fault-tolerance, distributed systems may also be the source of many failures. [6]. Therefore, a vigilant design of failure recovery is the next steps to achieving more availability in distributed system. The ability of the system to recover itself directly boosts availability in the sense that an outage can automatically be fixed without hurting user experience or causing significant loss. Time and correctness are the most important

criteria in this process. They are contributed by the two prerequisites for failure recovery within redundant system; fault detection and replication.

A prompt and accurate failure detection with minimum time to recover is a critical factor in providing high availability in distributed system. Failure detection and recovery has become an active research area due to the needs that arise with more and more complex system. Failures are arising due to the inherently unreliable nature of distributed environment. The diverse nature of Grid Computing for example, requires the inclusion of fault tolerance capability not as a supplementary feature but rather a prerequisite [7]. As for cloud computing, it has become a critical issue due to its nature of complex multi layers and the most of current approaches, fault tolerance is exclusively handled by the provider or the customer which leads to partial or inefficient solutions [8].

In this research, a stand-alone system for National Bioinformatics System or NABTICS will be the case study of how the integration into a specific distributed environment can improve its availability by providing it with fault tolerance feature. NABTICS is a knowledge repository system that is aimed to provide bioinformatics tools resource that specifically focus on marine organisms. NABTICS is the initative of Marine Biotechnology Institute of Universiti Malaysia Terengganu. The process to approach faults in this project is entailed by a number of activities. A background monitoring is established

to check member nodes heartbeats to confirm their liveness. At the same time a critical adaptive threshold is maintained to make sure a failure is detected within it. A confirmed failure will set the recovery mode where a selection process is started before a fail-over server is instructed. In effect, the service is continued as the fail-over masks the recent failure.

1.2 Problem Statement

1. A unified single server system lacks redundancy due to there is no extra hardware to be a backup if the primary server has problem. This problem could jeopardize system reliability where data could be lost and service could be interrupted without recovery. Whereas having regular backup is feasible, it is not efficient because recovery is not autonomous. A system like NABTICS involves application to process data and data center like services. If the system is unavailable, the work of the researchers could be interrupted and can cause congestion when the system come back online.

2. As researchers experience progressive findings and new discovery in the Microbial fields, a need for a highly reliable and available software arises as well as more functionalities. The new vision not only eases the task of the researchers but also contributes to the research advancement of Microbes study and Marine Ecology. Researchers will be able to

better manage their research database as well as conveniently share with other researchers. Adding more functionalities to the system will be more meaningful if at the same time the system can provide high availability.

3. Fault tolerance is an important attribute to achieve more operational time i.e. higher availability. There are many solutions to fault tolerance that involve either hardware, software approaches or the combination of both. While focusing only on hardware approaches can be more costly, the lack of software consideration can also threaten efficiency in recovery where more time is taken or incorrectness can occur. Therefore a good approach should employ as much as possible the resources that are available and use a superior technique for fault detection and replication.

1.3 Objectives

The main objectives in this research can be summarized as the following:

1. To build a server cluster for NABTICS modules thereby creating a distributed environment

2. To introduce redundancy through replication which will improve the overall availability of the system

3. To build a framework where fault tolerance and system performance can be achieved through:
   - Monitoring and indexing services that check the liveness of the components and record their statuses
   - A fault detection technique that is complete, accurate and timely
   - A recovery process that is autonomous with low latency
4. To analyze the performance of the new implementation

1.4 Research Scope

This research focuses on delivering continuous service of NABTICS system by continuous monitoring of the server cluster components and prompting a recovery process when failure is detected and confirmed. The fail-stop failure model is assumed in this research.

1.5 Contributions

The contributions in this thesis are:

1. Developed a monitoring program that can be used to monitor server from other site. The program uses heartbeat interaction policy on socket

connection and can be used on Linux and Windows system.

2. Developed a method to achieve fault tolerance that is a composition of several other techniques in fault detection and neighbor replication. The method can timely detect a site failure and confirm it. Subsequently a recovery process will presume and restore the system back online for users. The process is autonomous and users do not experience a significant downtime.

3. Created a distributed environment within a server cluster. The modules of the NABTICS system are placed and served from multiple sites but is accessed via one proxy. Having multiple components effectively reduce probability of unavailability. Furthermore, it is made possible to have multiple replicas strategically among the servers thus introducing redundancy and increasing availability of the system modules.

1.6 Thesis Organization

The work presented is organized in six chapters. The rest of the chapters after this chapter is organized as the following.

– **Chapter two** describes preliminary concepts and related methodologies. The concept of availability is disccused. The terminologies in High

Availability studies are also presented. It also reviews some existing literature on fault detection and recovery techniques. The characteristics and attributes of system faults are also studied. From this the discussion leads to the metrics and QoS of fault detection. The distinctions between load balancing and fail-over clustering are also highlighted. All the discussions become basis to the methodology of this thesis' project. Providers' solution for High Availability systems are also discussed to keep up with current trend and technologies.

– **Chapter three** proposes a methodology for the work that involves modularization of NABTICS for a server cluster, the monitoring technique and fault detection implementation, and lastly the fault recovery with neighbor replication. A technique for database replication is also proposed. The methodology of this research is divided into two phases where the first phase includes a parallel development of preparing NABTICS into distributed platforms and implementing and testing the High Availability framework. In the second phase, it involves integrating both of the work.

– **Chapter four** steps into the details of the implementation, environment setups and running the system based on the discussion in Chapter two to demonstrate the new framework for NABTICS application. The implementation uses the standard LAMP (Linux, Apache, MySQL, PHP) stack for the web application side and socket and SSH communication

for the middleware layer.

- **Chapter five** presents quantitative and qualitative results and discussion of the system performance and limitations.

- Finally, **chapter six** concludes the research and discusses possible future works in regards to this research.

Chapter 2

LITERATURE REVIEW

2.1 Introduction

This chapter describes the related background of this thesis' project and its application. First of all, the concept of availability is studied. The terminologies in High Availability studies are also presented. It also reviews some existing literature on fault detection and recovery techniques. The characteristics and attributes of system faults are also studied. From this the discussion leads to the metrics and QoS of fault detection. The distinctions between load balancing and fail-over clustering are also highlighted. All the discussions become basis to the methodology of this thesis' project. Providers' solution for High Availability systems are also discussed to keep up with current trend and technologies.

2.2 Availability and Unavailability

A wide range of availability classifications and definitions exist. It is largely based on the types of downtimes used in the computation and on the relationship

with time. According to ANSI [9], the availability of a system may be defined as the degree to which a system or a component is operational and accessible when required for use.

Many different works have also considered availability as an attribute of dependability. By adding the notion of recovery, system availability is branched from the concept of system reliability. According to Gokhale et al. [1], a crucial difference between reliability and availability is that reliability refers to failure-free operation during an entire interval, while availability refers to failure-free operation at a given instant of time.

In the article "Standardizing Availability Definition", 1995 [10] it states that, if one considers both reliability (probability that the item will not fail) and maintainability (the probability that the item is successfully restored after failure), then an additional metric is needed for the probability that the component/system is operational at a given time, t. This metric is availability.

Schmidth [11] defined availability as the frequency or duration in which a service or a system component is available for use. The base availability measurement is the ratio of uptime to total elapsed time [11].

$$Availability = \frac{Operational}{Operational + Non-operational} \tag{2.1}$$

In availability engineering and availability studies, unavailability values

are generally used as compared to the availability values. According to Reliability Prediction Basics by ITEM Software, 2007 unavailability or $Q(t)$ is the probability that the component or system is not operating at time t, given that is was operating at time zero.

Conversely, availability, $A(t)$, represents the probability that the component or system is operating at time, t given that it was operating at time zero. Both $Q(t)$ and $A(t)$ has numerical values from 0 to 1 and has no units (ITEM Software Inc, 2007). The unavailability, $Q(t)$ can also be defined as the component or system probability is in the non-functional state at time t and is equal to the number of the non-functional components at time t divided by the total sample. Since a component or system must be either in the operating or non-operating state at any time, the following relationship holds true:

$$Unavailability, Q(t) = 1 - Availability, A(t) \tag{2.2}$$

In this relationship, the probability of availability with the absent of unavailability can be calculated. Both parameters can be used in availability assessments, safety and cost related studies.

2.3 Mean Time Between Failures (MTBF)

Reliability of repairable items can be measured with Mean Time Between Failures (MTBF). MTBF basically refers to the amount of time passed before

a component, assembly or system fails, when subjected to constant failure rate. Or it is simply the expected value of time between two consecutive failures. For constant failure rate systems, MTBF can also be calculated as the inverse of the failure rate, λ.

2.4 Mean Time To Failure (MTTF)

Mean time to failure (MTTF) is used to measure the reliability of non-repairable systems (ITEM Software Inc, 2007). It represents the expected mean time before the occurrence of the first failure. For constant failure rate systems, MTTF is the inverse of the failure rate λ. If failure rate λ, is in failures/million hours, MTTF = 1,000,000 / λ.

Typically, MTBF is applicable to components that could be repaired and returned to service whereas MTTF applies to parts that would no longer be used upon failure. However, MTBF can also be used for both repairable ad non-repairable items.

2.5 Mean Time To Repair (MTTR)

Mean Time To Repair (MTTR) refers to the duration of time between failure and completion of any corrective or preventative maintenance repairs (ITEM

Software Inc, 2007). The term only applies to repairable systems.

2.6 Probability of availability

The goal of failure detection and failure recovery study is to reduce the sudden unavailability (MTTR) so that computer systems can improve availability. Based on equation 2.1, the probability of availability can be expressed as:

$$Availability = \frac{MTBF}{MTBF + MTTR} \tag{2.3}$$

Availability of a system can also be referred to as the probability that a system will be available over a time interval T [12]. In other words, availability is a conditional probability that a system survives for the time interval [0,t], given that it was operational at time t_0. Jia and Zhou [12] have also expressed availability in the ratio of operational nodes to total number of nodes. Similarly, unavailability was expressed in the ratio of failed nodes to total number of nodes.

2.7 Failure Rates

The probability of availability is based on failure rates. Every product has a failure rate, λ which is the number of units failing per unit time. Conditional

Failure Rate or Failure Intensity $\lambda(t)$ provides a measure of reliability for a product. ITEM Software Inc.(2007) defined $\lambda(t)$, as the expected number of times an item will fail in a specified time period, given that it was good at t_0 and is working at time t. A failure rate of 0.2%/1000 hours or 2 failures per million hours (fpmh) or 500,000 hours until a failure can be expressed as:

$$0.2/100 * 1/1000 = 2/10^6 = 2fpmh$$

By considering a node with 0.2% of failure per 1000 hours, the probability of failures $Q(t)$, (sudden unavailability) per year could be calculated as:

$$Q(t) = 0.2/100 * 1/1000 * 24 * 365 = 0.01752$$

Since availability is given by $A(t) = 1 - Q(t)$, therefore $A(t) = 1 - 0.01752 = 0.98248$ In three years the unavailability is:

$$Q(t) = 0.2/100 * 1/1000 * 24 * 365 * 3 = 0.05256$$

Therefore, the availability for three year is:

$$A(t) = 1 - 0.01752 = 0.98248$$

Based on this equation, it can be calculated that in 3 years (26,280 hours) the availability, $A(t)$ is approximately 0.95. This means that if such a unit is operational 24 hours a day for 3 years, the probability of it surviving that time is about 95%. The same calculation for a ten year period will give $A(t)$ a value of about 84%.

2.8 System availability

According to Romeu [13], independent components are those whose failures do not affect the performance of any other system component. A series system is a configuration such that if any one of the system components fails, the entire system fails. A parallel system is a configuration such that, as long as not all the system components fail, the entire system still works. System reliability and availability are assessed by structuring the system as an interconnection of components in series and parallel.

2.8.1 Availability in Series

Two parts, X and Y are considered to be operating in series if failure of either of the parts results in failure of the combination. For this combined system, it is only available if both Part X and Part Y works. Hence, the serial availability of the combined system is given by the product of the two parts as shown in

the following equation.

$$A = A_x * A_y \tag{2.4}$$

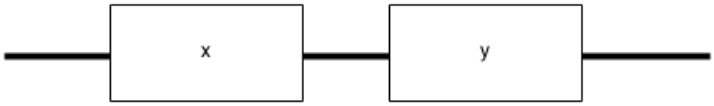

Figure 2.1: Availability in series

Based on the above equation, the combined serial availability of two components is always lower than the availability of its individual components.

2.8.2 Availability in parallel

Two parts, X and Y, are considered to be operating in parallel if either part is available. Only when both parts fail, the combination is considered failed. Hence, this combination enables the design of a highly reliable system which makes it suitable for mission critical systems. Equation 1.6 gives the availability for parallel systems.

$$A = 1 - (1 - A_x) * (1 - A_y) \tag{2.5}$$

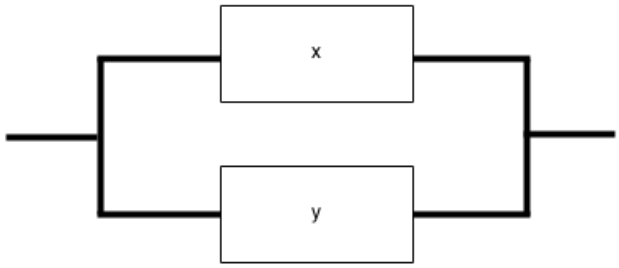

Figure 2.2: Availability in parallel

2.8.3 Availability in joint parallel and series

In real environment, it is common to have two or more sets of parallel components connected in series. Referring to Figure 1-3, the availability can be defined as:

$$A = (1-(1-A_x)*(1-A_y))*(1-(1-A_w)*(1-A_z)) \qquad (2.6)$$

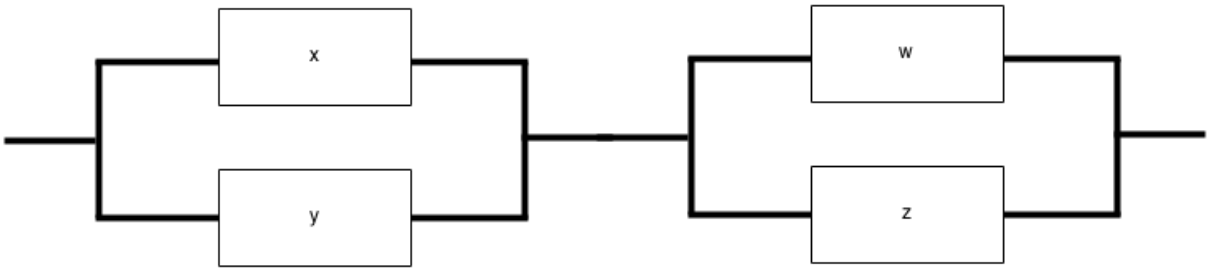

Figure 2.3: Availability in joint parallel and series

2.8.4 Availability in distributed system

In the parallel distributed system, the system works unless all nodes fail. Connecting machines in parallel contributes to the system redundancy enhancement.

Let A = availability, Q = unavailability, then the system unavailability as given by [14] is as follows:

$Q = Q_1 * Q_2 * Q_3 * ... * Q_n$

$Q = (1 - A_1) * (1 - A_2) * (1 - A_3) * ... * (1 - A_n)$

Thus, the availability of the distributed parallel system can be calculated as:

$A = 1 - Q = 1 - Q_1 * Q_2 * Q_3 * ... * Q_n$

$A = 1 - Q = 1 - (1 - A_1) * (1 - A_2) * (1 - A_3) * ... * (1 - A_n)$

$$A = 1 - \prod_{i=1}^{n} (1 - A_i) \tag{2.7}$$

Availability in joint parallel and series system where each ith component has the same unavailability is given in equation 1.10.

2.9 Terminology

The following terms are often used in High Availability discussion.

1. **Node**

   Any computer (real or virtual) that is a member of a cluster and invisible to the user.

2. **Floating IP address**

   A floating IP address is similar to a virtual IP address. It is always assigned to the active node. Users should always access this address so they do not have to know which node is active.

3. **Continuous operation**

   Continuous operation is an uninterrupted or non-disruptive level of operation where changes to hardware and software are apparent to users. Planned outages normally take place in environments that are designed to provide continuous operation. These kinds of environments are designed to avoid unplanned outages.

4. **Continuous availability**

   continuous availability is an uninterrupted, non-disruptive, level of service that is provided ti users. It provides the highest level of availability that can possibly be achieved. Planned or unplanned outages of hardware

or software cannot exist in environments that are designed to provide continuous availability.

5. **Failover**

   failover is the procedure in which one or more node resources are transferred to another nodes or nodes in the same cluster because of failure or maintenance.

6. **Failback**

   failback is the procedure in which one or more resources of a non-functional node are returned to its original owner once it becomes available.

7. **Fencing**

   Describes the concept of preventing access to a shared resource by isolated or failing cluster members. Should a cluster node fail, it will be shut down or reset to prevent it from causing trouble. This way, resources are locked out of a node whose status is uncertain.

8. **Primary (active) node**

   A principal or main node is a member of a cluster, which holds the cluster resources and runs processes against those resources. When the node os conciliated, the ownership of these resources stops and is passed to the standby node.

9. **Standby (secondary, passive, or failover node)**

   A standby node, also known as a passive, secondary or failover node

is a member of a distributed system that is able to access resources and running processes. However, it is in a standby position until the principal node is conciliated or has to be stopped. At that point, all resources fail over to the standby node, which becomes the active node.

10. **Multicast**

    A technology used for a one-to-many communication within a network that can be used for cluster communication.

11. **Single point of failure**

    A single point of failure (SPOF) exists when a hardware of software component of a system can potentially bring down the entire system without any means of quick recovery. Highly available systems tend to avoid a single point of failure by using redundancy in every operation.

12. **Cluster**

    A cluster is a group of nodes and resources that act as one entity to enable high availability or load balancing capabilities.

13. **Outage**

    for the intention of this thesis, outage is the failure of services or applications for a particular period of time. An outage can be planned or unplanned:

    - Planned outage

      planned outage takes place when services or applications are interrupted

because of planned maintence or changes, which are expected to be reinstated at a specific time

- Unplanned outage

  unplanned outage takes place when services or applications are interrupted because of events that are out of control such as natural disasters. Unplanned outages can also be caused by human errors and hardware or software failures

14. **Uptime**

    uptime is the duration of time when applications or services are available

15. **Downtime**

    downtime is the duration of time when services or applications are not available. It is usually calculated from the time that the outage takes place to the time when the services or applications are available.

16. **Service level agreement**

    Service level agreements (SLAs) ascertain the degree of responsibility to maintain services that are available to users, costs, resources and the complexity of the services.

2.10 Behavior of Failed System

In distributed systems, failure is a normalcy and should be anticipated. The following are classes of system failures.

- Byzantine

  Byzantine is arbitrary failure which is difficult to solve. A byzantine system is one that does not stop after a failure but instead behaves in an inconsistent way. It may give wrong information or respond late to an input or trigger. [4] implies that Byzantine system lies about what they know. Compared to other type, there has not been many work in this type of system.

- Fail-fast

  Fail-fast systems are usually designed to stop normal operation rather than attempt to continue a possibly flawed process. It behaves like Byzantine system for a short time but moves into fail-stop mode soon. A fail-fast module passes the responsibility for handling errors, but not detecting them, to the next-higher system design level. In a fault-tolerant system by means of redundant components, the fail-fast individual components can give the system enough information to successfully tolerate a failure as opposed to fail-stop where the system will perform a fail-over as recovery.

- Fail-stop

  Also known as crash-stop, the fail-stop model is one in which once it has failed it halts and does not execute any other operation. On contrary in crash-recovery the process halts after failure and then recover after a while [15]. Fail-stop models are often used in computer systems where correctness is very important since it can prevent any incorrect behavior. It does not matter what type of faults or failures that have caused this behavior but it is necessary for the system to not perform any operation after it has failed.

  In this thesis, the fail-stop non-byzantine model is considered.

2.11 Failure Detection

Failure detection is a process in which information about faulty nodes is collected [16]. This process involves isolation and declaration of a fault to enable proper recovery actions to start.It is a prerequisite to failure recovery in distributed system.

2.11.1 Properties

Chandra and Toueg [17] defines eight classes of failure detection. They are metricized by the following two properties:

- Completeness

  Strong completeness is marked when every faulty process is eventually permanently suspected by every non-faulty process. Weaker completeness means that every faulty process is eventually permanently suspected by some non-faulty process. Completeness is quantization of the non-faulty process that act as suspectors.

- Accuracy

  Accuracy refers to the degree of suspicion a system can impose on a non-fault processes, or faulty processes that have not crashed yet. Strong accuracy marks the system never suspects any process before it actually crashes.

In other words, a complete fault detector suspects a faulty process with conviction and with consistency among its processes, and an accurate fault detector do not place any suspicion on a non-faulty process until a fault actually happens affirmatively. The degrees of this metrics are varied resulting in eight classes of failure detection [17].

2.11.2 Quality of Service (QoS)

Chen et al. defined the following QoS metrics for a failure detectors in a fail-stop system in [18]:

Detection time (T_D):

The time interval between the crash of the process and the time in which the failure detector starts to suspect the process in a permanent way; this metric quantifies the delay of the failure detector.

Mistake recurrence time (T_{MR}): The time between two successive mistakes.

Mistake duration (T_M): The time the failure detector takes to correct a mistake; it measures the time interval between an erroneous detection and its correction.

The completeness attribute of fault detection is referred to T_D, while the accuracy attribute is related to T_{MR} and T_M. The query accuracy probability, P_A is derived from the three metrics. P_A is defined as: $P_A = \frac{T_{MR}-T_M}{T_{MR}}$. It represents the probability that the output of the failure detector is correct at a random time. It corresponds to the classical availability concept [18].

2.12 Approaches to Failure Detection

With more hardware the probability goes up that there will be a failure somewhere. Adding more software and the complex interactions between different programs creates greater chance for more bugs, including the unusual ones. As a result,

any distributed design will carefully consider failure and diagnostic scenarios [19].

There has been normally two interaction policies for detecting failures:

- **Heartbeat**: A node will send out messages every designated time period to alert others that it is still alive. The heartbeat model or push model is the most common technique for monitoring crash failure. Many state-of-the-art failure detectors are based on heartbeats. In this model the direction of control flow matches the direction of information flow. In addition, the model has active monitorable objects. These objects will periodically send heartbeat messages to inform other objects that they are still alive. If no heartbeat is received by the monitor within the specified time bounds, it starts suspecting the object. since only one-way messages are sent in the system, this method is efficient. If several monitors are monitoring the same objects, the model may be implemented with hardware multicast facilities. The abstraction of the roles of objects involved in a monitoring system is performed by three interfaces namely monitors, monitorable objects and notifiable objects. monitors basically collect information about component failures.

– **Pinging**: A node will ask other nodes whether they are alive, and if they reply in some timely manner, then the original node is satisfied that the other node is alive. In this model the flow of information is in the opposite direction of the control flow i.e. only when requested from controller. compared to push model, monitored objects in pull model are passive. the monitors periodically send live status request to check on the monitored objects. if monitored object replies, it means that it is still alive. since two-way messages are sent to the monitored objects, this model is regarded as less efficient and less popular than the push model. however, the pull model is easier to use because the monitored objects being passive does not require to consider the frequency monitor expects to receive the message.

2.12.1 Affirmative Adaptive Failure Detection (AAFD)

In [20], an adaptive technique for failure detection was introduced. This technique incorporates pinging to ensure the liveliness of a node once it is suspected for failure thus is aptly called affirmative. This technique performs a central sampling on the heartbeat inter-arrival time to obtain the estimation for the next heartbeat arrival. If the next heartbeat did not arrive within this timeframe, the detector raises a state of suspicion and sends a ping echo request to the monitored node. The threshold for the heartbeat to arrive

reflects the current state of the node CPU load and network condition.

$$H_{max} = \sum_{i=1}^{n} S_i + S_{n+1} \tag{2.8}$$

where,

- H_{max}, Maximum HB message arrival time
- $\sum_{i=1}^{n} S_i$, Total time elapsed (total heartbeat time before the last heartbeat)
- $$S_{n+1} = \frac{\frac{\sum_{i=1}^{n-1} S_i}{|S_{n-1}|} + S_n}{2} \tag{2.9}$$

  where,

  - $\sum_{i=1}^{n-1} S_i$ is the total heartbeat time in S_{n-1}
  - $|S_{n-1}|$ is the size of the sampling in S_{n-1}
  - S_n is the inter-arrival of the most recent heartbeat arrived

Considering again, p is monitoring q and is waiting for the next heartbeat $(n+1)$ from q. The probability of the $q(n+1)^{th}$ heartbeat is influenced by the last heartbeats. From the analysis of heartbeat inter-arrival time, the last heartbeat S_n has a significant likelihood to resemble the next heartbeat S_{n+1} therefore is factored by 50% for the next heartbeat while the rest of in sample S is factored by 50%. This can ensure a close reflection of the current condition of the monitored node and network. A deviation can be detected based on this

reflection and will be confirmed by pinging to make sure it is a permanent failure instead of a temporary glitch that occasionally happens due to network or CPU load.

2.12.2 Gossip-style Failure Detection

Gossip-style failure detection was first introduced in 1998 by van Renesse et al. [4]. The basic protocol in concept is each node sends out some data to a random node. Data propagates through the system node by node and eventually build a global map from local interactions reducing the message counts from polynomial to linear [21]. The receiver reconciles the membership lists by taking whichever version number is greater. If after a fail time period there has been no updated heartbeat number for a particular node, then that node is deemed failed.

R. Subramaniyan et al. [22] introduced GEMS, a gossip-enabled monitoring service. Their technique uses gossip protocols to detect failures in large, distributed systems in an asynchronous manner without the limitations of reliable multicasting in group communications. GEMS is a highly scalable technique being able to detect network partitions and dynamic insertion of new nodes. Using the very simple methodology, the technique combines reachability data from a lot of different nodes to quickly determine if a node

is down. Other than for failure detection, the technique also use gossip as a form of messaging making an abstract communication from application level code instead of individual module connection.

2.13 Load-balancing and Fail-over Clustering: A Comparison

Load-balancing and Fail-over Clustering are often interchangeable. This section tries to distinguish between these two techniques. First of all, the techniques differ in their purposes although both are useful in failure recovery and increasing availability. Load-balancing is focused on improving performance by splitting a service and spreading load across multiple servers while fail-over clustering is more focused on increasing availability where a failed server is replaced by another working server within the cluster.

The common techniques in load-balancing is by having multiple front-ends where the client is served by random IP of these front-ends. This is effective to reduce load and increase performance. In this technique a new server can easily be added to the resource pool and traffic is immediately decided and sent by the load-balancer to all servers thus it is highly scalable. Availability is increased in the sense that the redundant front-end servers can be backup to a failed server as decided by the load balancer.

In fail-over clustering, replication is a common technique where servers maintain replicated copies of each other. At one time, there may only be one server running. Maintenance is also possible by taking the necessary servers offline while having other server do its job meanwhile.

The load-balancing technique is often used for front-end servers while the fail-over clustering technique is more often used for back-end servers. The combination of both techniques can complement each other and increase both performance and availability of each other and effectively of the entire system.

2.14 Failure Recovery

In distributed environment, failures can become a normalcy. A failure recovery procedure is required to restore the system to its functional state thus ensuring High Availability. Redundancy is commonly used to eliminate SPOF and is a notorious technique for failure recovery in distributed environment. Basically there are three techniques used for redundacy.

1. Checkpointing

   Periodically saving the state of a process running in a computing resource so that in the event of failure, it can resume on a different resource.

2. Replication

   Maintaining copies or replicas of the processes on different sites executing in parallel so that at least one replica is alive.

3. Rescheduling

   Finding different resource to rerun the failed process from the beginning.

2.15 Failure Recovery using Replication Techniques

Replicating components is the answer to achieve high availability in distributed system. In replication based recovery technique, there must be multiple copies of the same object (replicas) that are running on parallel. If one replicated object fails there will be another replica that will take over it without having to take the system offline. Replication id the primary goals in designing a dependable distributed system [23] emphasized maintaining the data on some replicas to provide reliable services. On the other hand, keeping all of the replicas updated requires extra communication and processing. Several techniques have been proposed for replicating data management with different reliability levels. They can classified into two categories; synchronous replication and asynchronous replication. The lack of global clock makes the asynchronous replication less precise but is less costly to deploy. The terminology to also consider is for active and passive replication in systems that replicate data or

services:

- Active replication is performed by processing the same request at every replica.
- Passive replication involves processing each single request on a single replica and then transferring its resultant state to the other replicas.

2.15.1 Two Replica Distribution Technique (TRDT)

Many classical approaches to replication are based on a primary/backup model where one device or process has unilateral control over one or more other processes or devices [24]. TRDT proposed by Shen et al., is composed of a primary and a secondary (which can become primary) nodes and is depicted in the diagram. In this technique, the nodes have identical storage capacity and all data has two replicas on different nodes and all nodes have two data replicas.

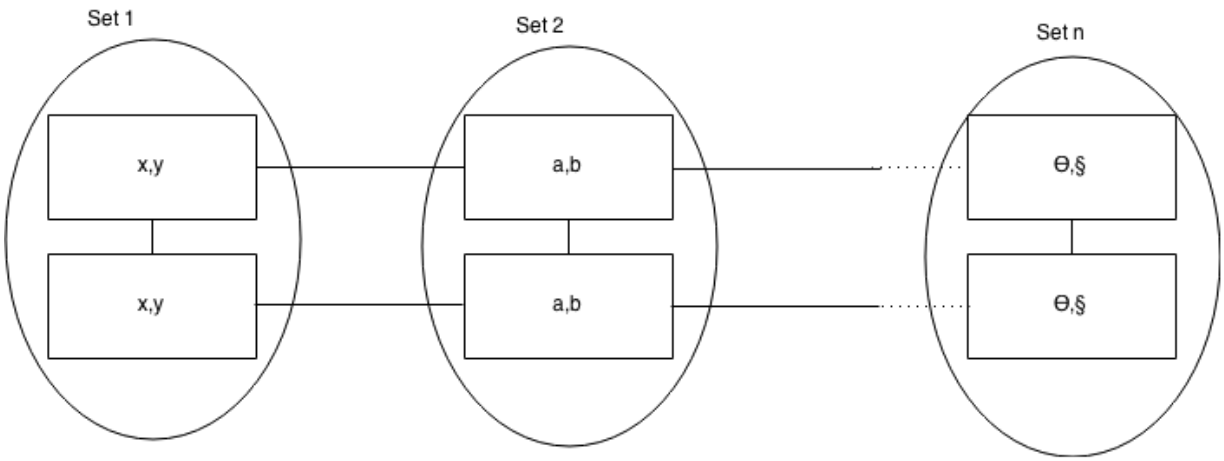

Figure 2.4: TRDT data replica distribution technique when $N = 2n$

Each box represents the node and each set consists of two nodes. Data x, y have two replicas in set 1 and so on until set n.

In this technique, replicator is the key component that performs the replication protocol. It builds a replication link between the primary and secondary replica with a log and storage for synchronization on individual nodes.

There are a number of projects that are based on this technique which include Linux-HA and Globus.

2.15.2 Neighbor Replica Distribution Technique (NRDT)

In NRDT all nodes are logically designed in the form of two-dimensional *nxn* grid structure [23]. If there are nodes in an environment where $N = n^2$, then

it will logically be arranged in the form of *nxn* grid. Each node has a primary data file while its adjacent neighbor contains a replica of its primary data file.

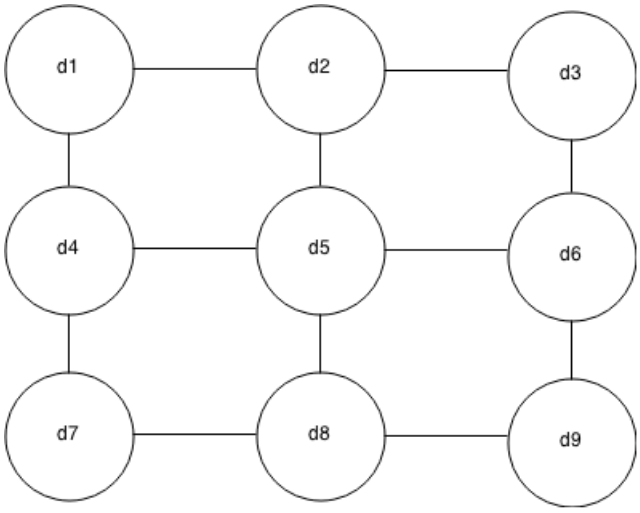

Figure 2.5: NRDT with 3 x 3 grid

A node X is a neighbor to node Y, if X is logically positioned adjacent to Y. A data will replicate to the neighboring nodes from its primary node. Using the property described in [23], the number of data replication, $m \leq 5$.

Let $N = n^2$ be a set of all nodes that are logically arranged in a grid form. Then all nodes are labbeled $n_{i,j}, 1 \leq i \leq n, 1 \leq j \leq n$. Two way links will connect nodes $n_{i,j}$ with its four neighbors, nodes $n_{i,j-1}$, $n_{i,j+1}$, $n_{i-1,j}$ and $n_{i+1,j}$ as long as there are nodes in the grid. Four nodes on the corners have only two adjacent nodes and other nodes on the boundaries have three neighbors. So the number of replication m is the number of maximum replicas a node can have and the primary data itself which is ≤ 5.

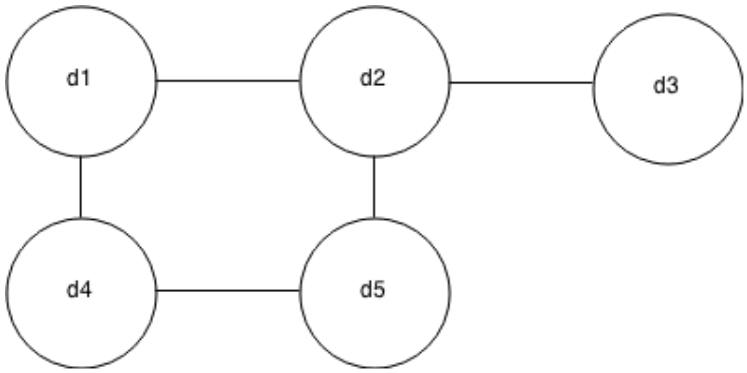

Figure 2.6: NRDT with 5 nodes

For the case of $N \neq n^2$, at most there is just one node that has only one neighbor. For $N = 5$, it can be structured as in figure 2.6. The property described for the number of replicas, m still holds true. In this case m is 2.

2.16 Resource Management System for High Availability

This section discusses briefly about available solutions for high availability systems. They differentiate each other in terms of architecture, features and type of computer systems they specialize for.

High Performance Computing usually requires scalability as large amount of nodes are involved such as in Grid [25]. Most monitoring system for High Performance computing uses hierarchical monitoring and replication

such as with multicast and peer to peer protocols (Ganglia by UC Berkeley and Astrolabe [26]) to cater with scalability. Though redundancy is easily heightened, these types of communications can introduce high overhead cost in both monitoring and replication which may not be worthwhile for scaled system. There are also many different solutions for resource monitoring for enterprise (Nagios (NAGIOS) Zenoss (Zenoss, Inc.) OpenNMS (The Open-NMS Group, Inc.) Safekit(Evidian)). They are usually designed for more heterogeneous computing as there could be different types of resources within the enterprise. The usage can be prone to error as their design is incorporated with many functions and is usually quite complex for average users.

Cloud-based monitoring provides monitoring as a service to customers. Similar to enterprise solution, cloud-based monitoring perform centralized infrastructure/application monitoring. However the installation and maintenance are provided for which can reduce the complexity. In term of cost, it might be more expensive than hosting a monitoring service inhouse especially for long term as cloud service usually charges per usage.

According [27], a high-availability stack serves one purpose: through a redundant setup of two or more nodes, ensure service availability and recover services automatically in case of a problem. Linux-HA is an active project and Pacemaker, which is branched from Linux Heartbeat is a popular open source High Availability Stack. The stack consists of four layers: storage, cluster

communications, resource management and applications. The Pacemaker's crm shell is a command interface that aims to provide simplified interface for resource management. There are many usages that involves integrating Pacemaker in Linux Heartbeat clusters.

2.17 Summary

As a summary, availability is an instantaneous measure of reliability and is related to the ratio of total time the system actually being operational over an entire operation time. From this evaluation, the probability of the system availability can be gathered for future consideration. Unavailability is a terminology often used in discussion in system engineering as a measure of downtime probability. Over the years, unavailability can increase proportionally.

MTBF is measured between each failure and is a direct indication of availability of a system. System that often fails has low MTBF. The goal for a system that aspires high availability is to have maximum MTBF. The robustness of hardware can only be extended so much. Therefore a technique that employs fault recovery is needed through both hardware and software implementation.

An implementation that enjoins serial and parallel connections can optimize

the availability of a system by decreasing the unavailability. Modules are connected in series in this system and directly gives unavailability to the system. By decreasing the unavailability of each module, the system total availability can be increased. This approach requires the employment of a replication technique that adds redundancy in the module.

Two implementations of failure detection are discussed that are Adaptive Affirmative Failure Detection (AAFD) and Gossip Enabled Monitoring System (GEMS). In AAFD heartbeat interaction is used to monitor the health of servers and an expection of heartbeat arrival is maintained to preemptively detect a failure and subsequently confirming it. In GEMS, a gossip approach is taken where a node randomly broadcasts its status to other nodes. The updates are kept and broadcasted to more nodes. After a certain time if there has been no updates from a certain node, the node is considered failed.

Fail-over clustering is a technique that enables backing up a service by IP take-over when the primary server has failed. It primarily uses the technique of replication. Load-balancing is a technique that spreads the load onto multiple servers to increase performance. In load-balancing availability is kept high by the multiple servers that are redundant.

The properties of failure detection technique are completeness and accuracy. A third property was added by Stelling *et. al* [15] that is timeliness. Timeliness

refers to how fast a fault can be detected in complete and accurate manner so that corrective action can be initiated soon. The Quality of Service for failure detectors are detection time, mistake recurrence time and mistake duration. These calculations can also be used to measure availability of a system.

A high availability framework often combines a resource monitoring to supervise sites, failure detection technique and recovery to gives fault tolerance to a system. A system with fault tolerance can withstand a number of failures and become more available therefore more reliable. In this chapter, a brief discussion on High Availability solutions by providers and open source projects for different computing is also given for knowledge about current technologies and trends.

Chapter 3

METHODOLOGY

3.1 Introduction

The objective of this research is on modeling a new technique to improve the availability of a knowledge based application called NABTICS by implementing it on top of a fault-tolerant framework. The methodology of this research is divided into two phases as shown in figure 3-1.

In Phase 1, the NABTICS are modularized to get independent modules to be distributed on multiple servers to create a server cluster. The modules are coupled with local replicas of the common database from the original version. The modules are tested to run via proxy to ensure connectivity. The neighboring relationship is also set by placing module replicas on immediate neighbor servers. The servers are stored with a replication of its immediate neighbors where in this implementation the are two adjacent neighbors for each server. In effect, a grid neighborhood is adopted. Finally connection to remote database is also implemented to ensure modules can send updates to a central common database. Parallely, a High Availability framework is being developed using several existing techniques discussed in chapter two and is

tested atomically to make sure the correctness of each implementation.

In Phase 2, the integration is completed and the final framework of the new model includes a server cluster, a proxy server, monitoring server and an indexing server as depicted in System Overview in figure 3-2. The server cluster is purposefully set up to create a distributed environment to enable parallel redundancy of node replication and the proxy is added for single view from the user side and for security. The High Availability framework is responsible to monitor the health of the nodes inside the cluster and command a failure recovery to be performed when a certain node has failed.

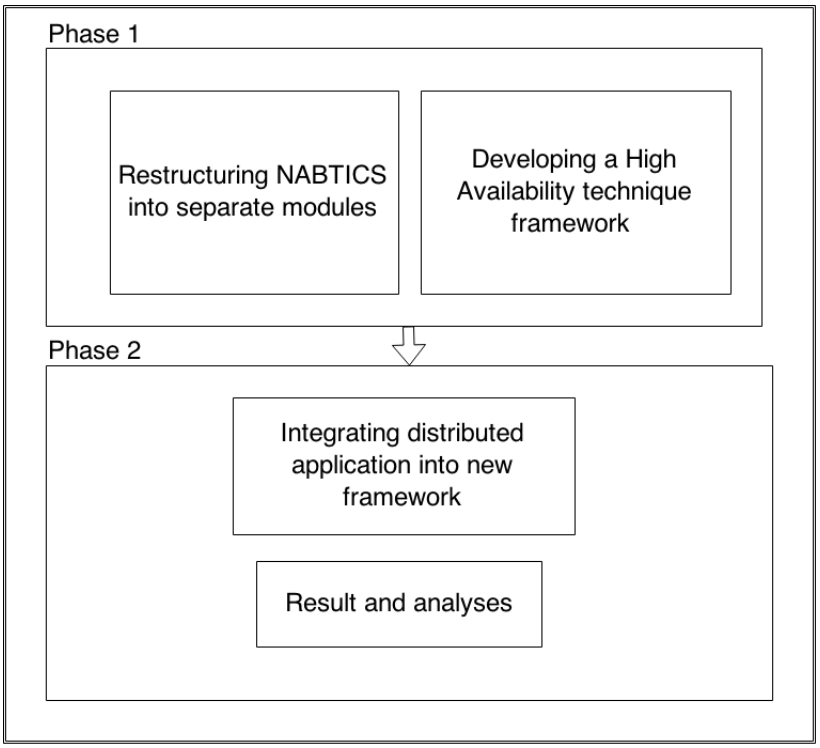

Figure 3.1: The Methodology Phases

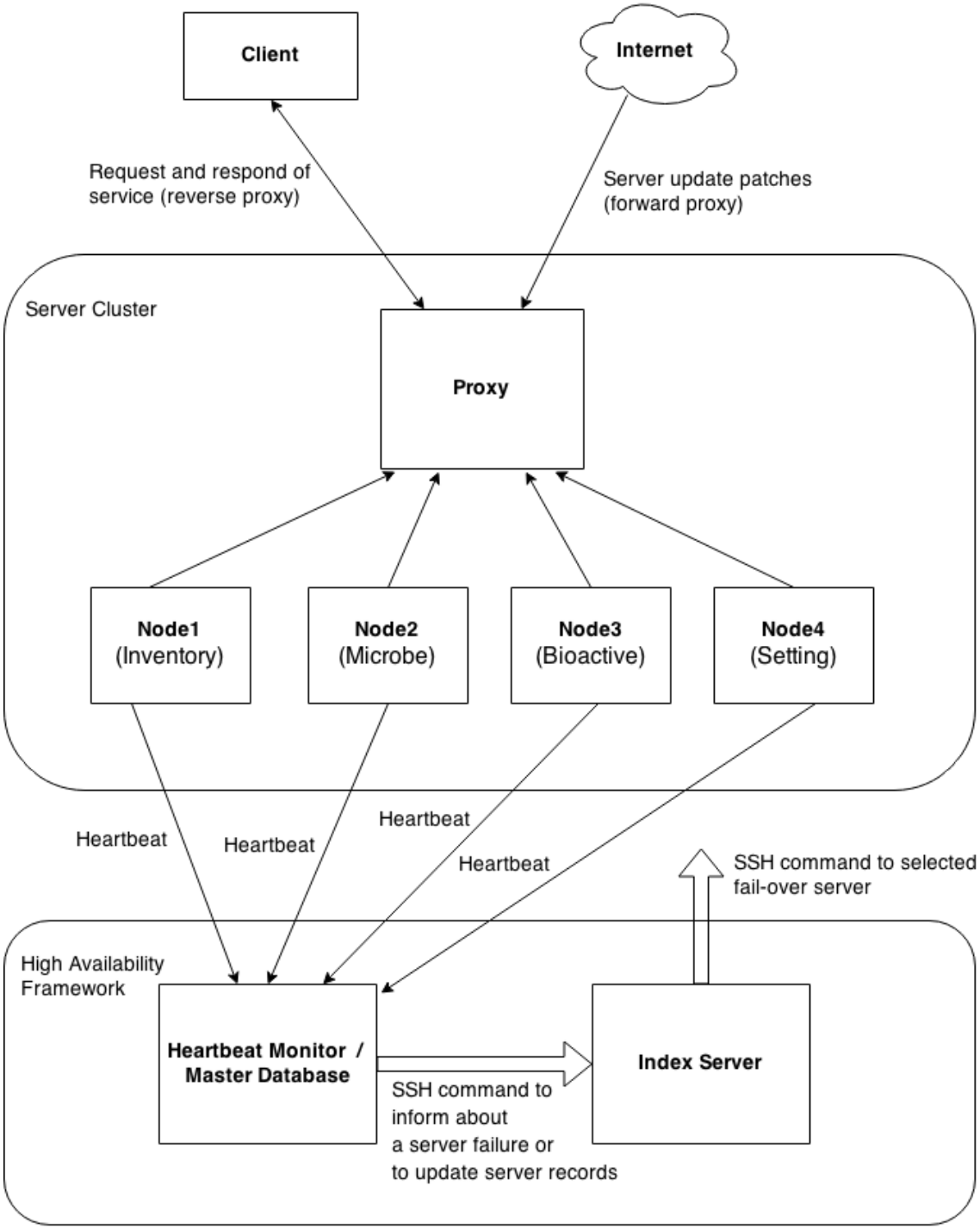

Figure 3.2: System Overview

This chapter will first discuss the modularization of a marine study application namely NABTICS from a stand-alone system into a distributed application in order to build a server cluster with high redundancy and subsequently discussion continues with the development of the resource monitoring framework for High Availability that performs the proper indexing and monitoring for the distributed application servers with capability for fault tolerance.

3.2 Server Cluster

A highly available system is very much desired. There are many approaches to attain high availability. The primary principle in these approaches is redundancy [28]. In this research redundancy will be achieved by having additional identical components in the system namely replicas. The multiple copies are standbys for backup in case a primary application server has failed to respond. This research utilizes the distributed application methodology to build a new version of NABTICS which will run on multiple servers inside a cluster. The purpose of having a distributed environment is to prepare a policy for replicated components. The replicas will be placed strategically on the associative application nodes.

Traditional applications relied on a single system to run them. Even in the client-server model, the application software had to run on either the client, or the server that the client was accessing [29]. However, distributed applications

run on both simultaneously. With distributed applications, if a node that is running a particular application goes down, another node can resume the task. This is meant by parallelism.

Using the LAMP stack (Linux-Apache-MySQL-PHP), the NABTICS modules were arranged to allow users of a computer network to access and use the services of NABTICS through a single, consistent user environment from a cluster.

Rearrangement of the NABTICS involves modest restructuring of the design and software. The goal is to have applications that are domain specific and uses remote database as well as a local database. Applying the modularity concept in Software Engineering, the applications were restructured to be independent from each other by making separate modules. Making it separate requires making all common points localized.

On client-end, the application is accessed through a proxy server. The proxy server is connected to the cluster nodes on a private internal network. Outside servers cannot communicate directly with the application nodes. This way the architecture of the distributed modules delivers transparency to clients where clients only query on a single IP despite the fact that the applications come from different sources. Additionally, the server also serves as a forward proxy where the application nodes can access the internet to get update patches.

In distributed application architecture, often middleware is used to enable interoperability between applications that run on different operating systems. In this research distributed system however, middleware is not used as on application level, the nodes are not required to communicate with each other. However a method of synchronization between the nodes on application level can be achieved by the synchronized data storage. It will be further discussed in coming sections.

The ability to handle failure allows server clusters to meet requirements for high availability, which is the ability to provide users with access to a service for a high percentage of time while reducing unscheduled outages [23]. Server clusters are designed so that the servers in the cluster work together to protect data, keep applications and services running after failure on one of the servers, and maintain consistency of the cluster configuration over time [30].

3.3 Neighbor replication

Referring to figure 3.2, the servers node1, node2, node3 and node4 are arranged logically in a grid structure where each two adjacent nodes are considered as neighbors. In this arrangement each of the NABTICS server has two neighbors. The purpose of this is to maintain a copy of each neighbor inside

every server so that they can serve as back up should a primary server for a certain module is failed. This copy is called replica which is the primary application of each individual server. The following table describes each server condition where it contains a primary module and two replica modules of its neighbors.

Node's name	Primary module	Neighbors	Replica modules
Node1	Inventory	Node2 Node3	Microbe Bioactive
Node2	Microbe	Node1 Node4	Inventory Setting
Node3	Bioactive	Node1 Node4	Inventory Setting
Node4	Setting	Node2 Node3	Microbe Bioactive

Table 3.1: The neighbor relationship of the nodes and the modules they contain

3.4 MySQL Database Replication

The stand-alone NABTICS is broken down into several applications that are independent of each other as already described in previous section. These applications are located on different servers which will comprise a cluster

as shown in figure 3.2. After NABTICS has been distributed into separate remote modules, singular database is no longer efficient because reading data from remote site will result in slower user experience. Furthermore, if the site that holds the data crash, data is irrecoverable unless there is other backup. But it still needs to be autonomous to not deviate from the goal of having prompt concealed failure recovery.

The other solution is to have each server have its own local database. It is more complex to implement but can promise faster access to data since database server resides locally. As NABTICS is an application of a lot of data and researchers need to collect data conveniently for their studies, the second option is more preferred. The challenge arises that several databases should be synced to ensure the data is identical and consistent. If this is not handled carefully, the consequence would be an unreliable application which will give wrong result to the researchers who subscribe to NABTICS. Basing on the arguments, in this research each application module has a local database for reduced cost of networking, faster read and more data reliability. In conjuction to each server local database for fast read, a master database as the fifth one is needed because all the local database needs to be synched with a master copy.

In this work, the proposed solution for database consistency is to make database replication. MySQL database replication provides a way to have

master-slave control on replicated databases [31]. In a master-slave relationship, master database will propagate all changes to its slave databases. This will ensure consistency in data as implied that only the master will hold the most correct version of the database. This will avoid many conflicts that could arise in multiple instances environment.

The heartbeat monitor (HBM) is predisposed to be the most reliable server as it is responsible for detecting every node's changes and for making all important decisions in this setting. Other than that, the HBM by design is behind any communication with the outside world therefore is considered most secure. Having this characteristic, HBM is chosen to host the master database. Database writes from the application node (as received by clients) is forwarded to HBM while the reads are done locally on application node. All changes on master will be propagated to the slave databases on asynchronous basis. If a slave server was lagging during an update, it can always update itself by checking for any updates in the master's log.

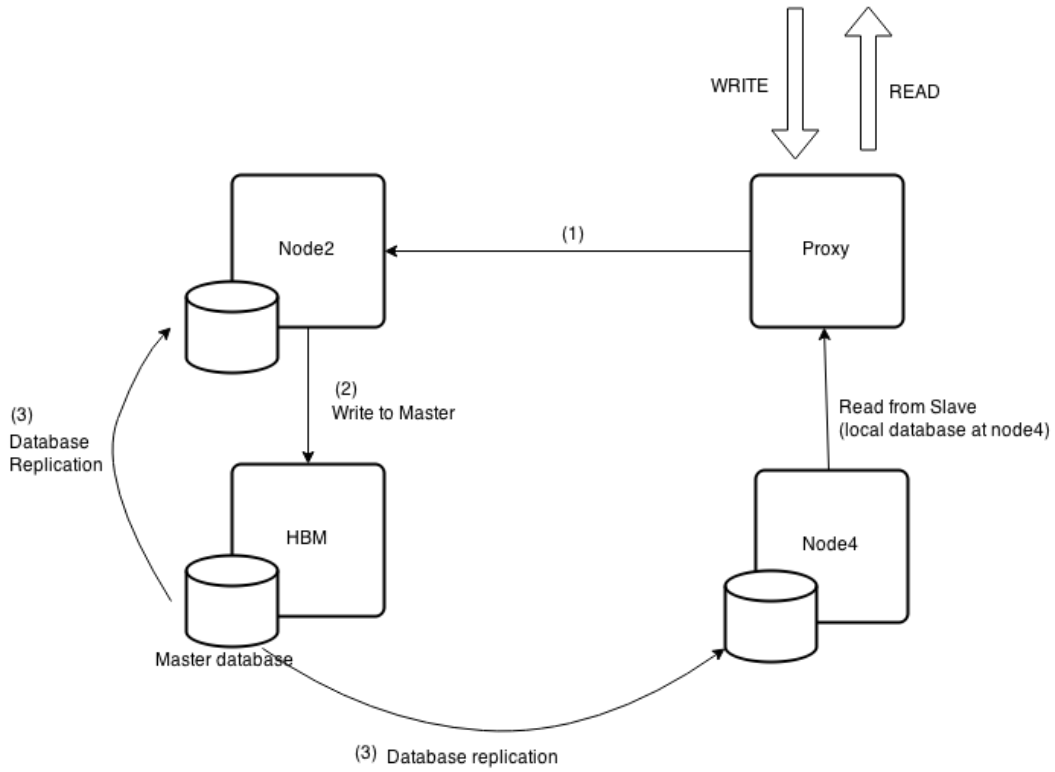

Figure 3.3: Database arrangement using MySQL Replication

Figure 3-3 illustrates the arrangement for database replication for the NABTICS modules. Following the sequence described in the illustration, the write is received on the Node2 module from the proxy. The Node2 module then accesses a remote database at HBM to write the new data. Because the databases at all nodes (in this figure only Node2 and Node4 databases are depicted) are set to be slave databases to master database at HBM, the new change to master copy are propagated to all slaves through database replication as shown in sequence (3). As a result, the new data is copied onto all servers and can be used by all modules. The read operation is also described in the

figure 3-3 where a read request is received by Node4 from the proxy. The module at Node4 then reads the local database and deliver the data to user that queries through the proxy. The same procedures if the user queries is received by Node2 who previously performed the data write to master database. Also at Node2, retrieving the same data is done locally. The mechanisms of MySQL Database Replication however is not included in this discussion but the settings are stepped through in chapter four. This arrangement delivers consistency since:

- Can avoid overwritten as only one copy gets all the first updates
- All database instances are updated simultaneously regardless of its version
- Faster reads since it is done locally

The redundant databases also contribute to increase parallelness in the system since it guarantees the modules to not be a single point of failure to others.

3.5 High Availability Framework

Redundancy by itself does not improve availability or reliability [5]. Multiple instances need to be entangled by having a proper monitoring and indexing system so that changes can be recorded and acted upon. [32]. In [20] the

Affirmative Adaptive Fault Detection (AAFD) component gives timely detection of node failure with completeness and high accuracy. Section 2.11.1 discusses the properties of failure detection techniques. The AAFD is designed to be dynamic by deploying an estimated time of arrival (ETA) threshold that adapts to the network and server condition as described in section 2.12.1. It can deliver higher availability by having an intuitive fault measure that can avoid false detection and enables a timely recovery. A false detection can trigger unnecessary recovery and put dispensable load on network and server which will result in waste of resources.

A service from primary NABTICS module can be lost during operation due to site crash or network failure. While this loss can be fatal to server application, it can also be due to temporary glitch. Non-affirmative fault detection may erroneously invokes unnecessary recovery. With affirmative action this can be avoided by only invoking the recovery once the faulty status of the server or network is confirmed. The affirmative step is enabled by employing 'ping' in the fault detection technique. After a timeout on waiting for incoming heartbeat, HBM sends a ping request from monitored node to confirm its status.

As seen in figure 3-2, the High Availability framework is consisted of a Heartbeat Monitor (HBM) and an Index Server (IS). The main task of HBM is to monitor the nodes inside the cluster closely down to an estimation of

the heartbeat inter arrival time. The tasks of the Index Server are to keep logs of the nodes' statuses and information, to perform neighbor selection process in case of a failure and to command a selected neighbor server to take over the failed server. Firstly, the monitored application nodes have to be registered at the Index Server. The nodes send periodic heart beat message to the Heartbeat Monitor to indicate its alive status. Data receiver socket in HBM accepts heartbeat messages. The heartbeat monitor program maintains a threshold for the next ETA of the heart beat based on the sampling of heart beat arrival time it receives. Based on the ETA, the HBM promptly can detect a faulty node and query the node late behavior by sending a ping request to it. If the node replies, HBM clears its suspicion and resumes monitoring with reset threshold. Otherwise if node is unreachable, HBM declares that it has failed and updates this information to Index Server and initiates it to perform recovery.

3.5.1 The C Socket Heartbeat Transaction

Heart beat generator is located at each node. HBM will supervise all sites based on their heart beats arrival time and completeness. The following figure presents the relationship of the HBM and application nodes connecting on socket communication and ping utility.

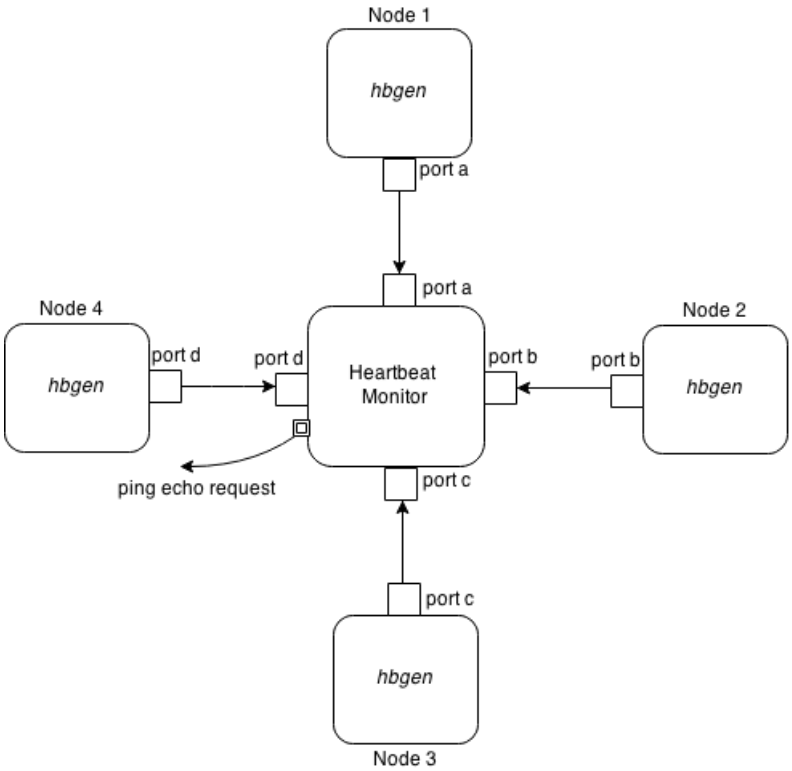

Figure 3.4: The portal operations at Heartbeat Monitor

The HBM uses different ports to monitor the nodes' heartbeat. These ports are dedicated to each node and are specified in the HBM programs. For ping echo request, HBM uses the default port for pinging as is specified by the Operating System since HBM uses the ping echo request that is a utility of the Operating System.

The heartbeat communication is designed on a C socket connection and a separate child process for each time a socket connection is to be established. Once it is permissible by the program that is when a socket is bound at its

port, a new process child will be created by $fork()$ system call. The child process will accept connection to active node socket and waits for a heartbeat message. Once the heartbeat is received, it will check the content of the message and gives signal if it has received the correct message. Parallelly a real timer was set outside the child process (in parent process) which gives interrupt signal when it has elapsed. If the system receives timer signal before the child process gives heartbeat signal, the TIMEUP flag will be set to initiate rechecking the node's condition and determining if recovery should be performed. Referring to figure 3-5, the heartbeat transaction is described. For each loop (in while loop), a new child process is created at HBM. Each time taken in each loop to wait for heartbeat message is stamped to maintain a representation of the average heartbeat inter-arrival time as described in section 2.12.1. The heartbeat message is sent by socket at application node and is received at HBM. The socket is closed after a heartbeat is received or after timer has elapsed. In the new loop, socket is reconnected again.

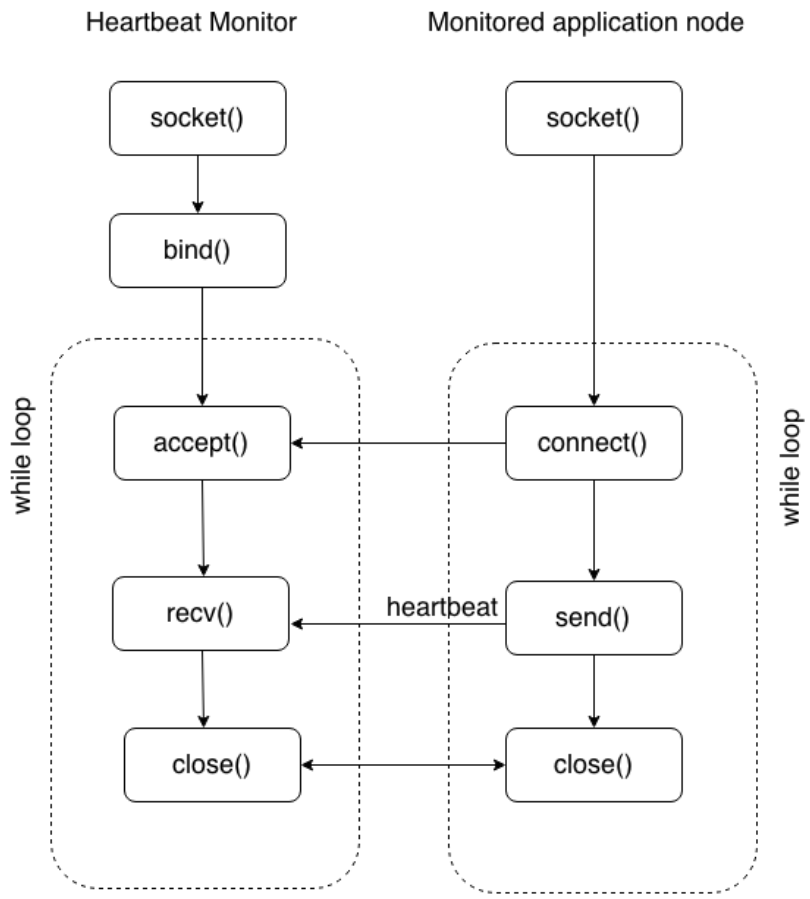

Figure 3.5: The C Socket for Heartbeat Transaction

The underlying fault detection capability involves a threshold for the heart beat. This formula was discussed in section 2.8. It is observed from the previous and most recent heart beat behavior. If the threshold is exceeded, the monitor will suspect a site failure. It then takes the precautionary action to confirm the site aliveness by sending a ping echo request to the suspicious node. After the status is confirmed to have failure it declares the failure to IS or otherwise it will continue monitoring with reset value of the threshold.

The full algorithm of heartbeat monitor and heartbeat generator are included in Appendix B.

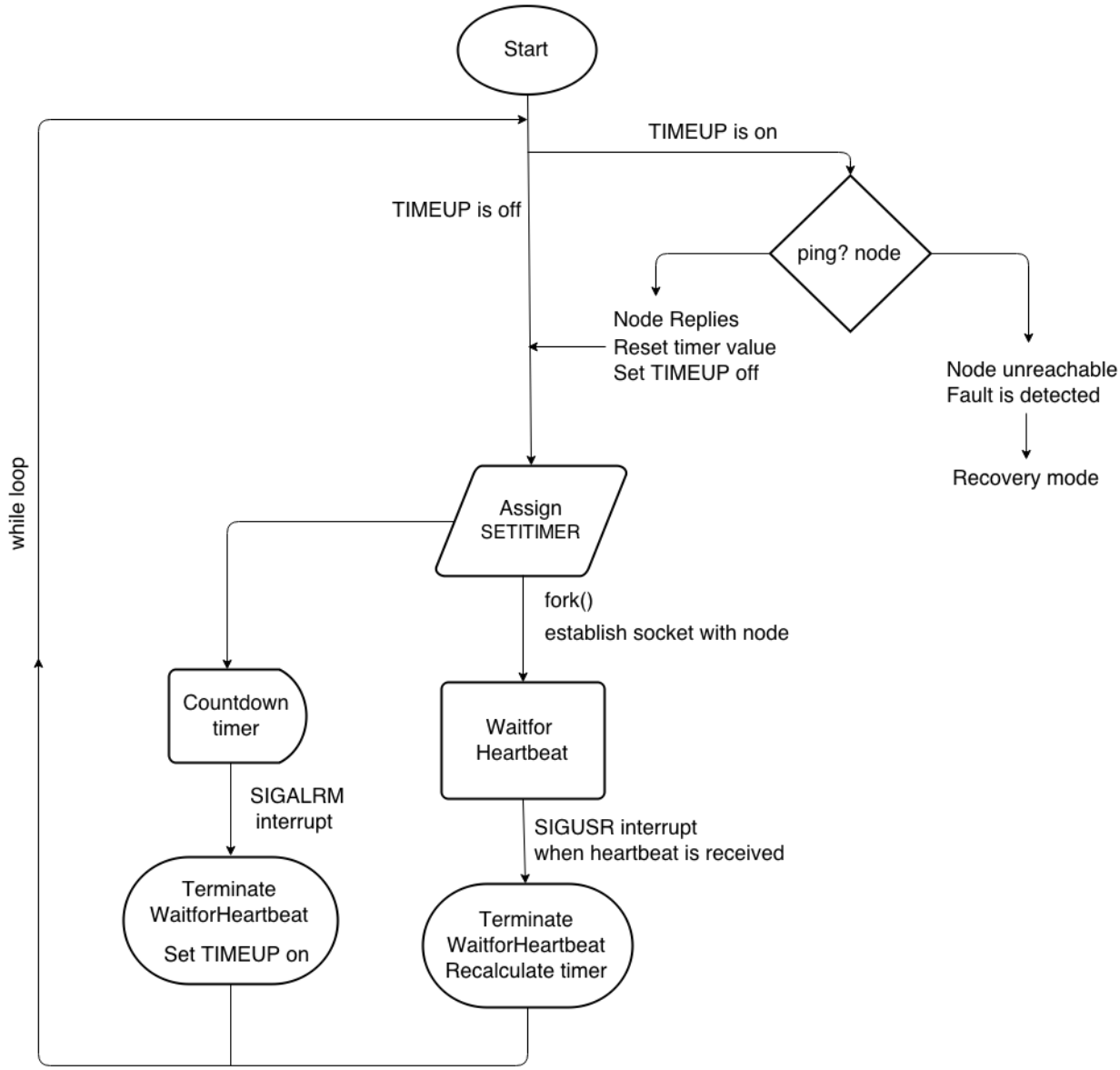

Figure 3.6: Flowchart of Fault Detection Process

The flowchart in figure 3-6 describes the flow of fault detection program. The initializing process is excluded in the flowchart. The flowchart describes

the process in the while loop. First of all in the loop the flag TIMEUP is polled to see if it is set indicating a timeup has happened in previous loop. The ping request is sent to the monitored node to confirm its status. If ping request returns node unreachable the failure is confirmed. At this point the program will enter recovery mode. If otherwise, ping reply is received the node is confirmed to be still alive and the HBM program will clear the TIMEUP flag indicating it is no longer a suspicion. The threshold value is reset to initial value to begin resampling.

If the loop is entered and TIMEUP flag is clear it means that previously the node was acting like expected i.e. no timeup did happen. In this case the program goes straight to assigning the timer with the threshold value. The threshold value was calculated in previous receive signal handler if it is not the initial value. Subsequently *fork()* is called to create a new child process. Inside the child process, socket connection is reestablished. After that it will listen indefinitely to the socket for heartbeat message. After a message is received it then checks if its the right heartbeat message. If it is indeed the right message, the child process sends a signal to parent process to make interrupt to program. The signal handler is entered and in here the time is stamped to obtain the heartbeat inter-arrival time. Also the threshold value is recalculated. If the heartbeat did not arrive in time, the timer will elapse and this will cause interrupt to program. The program will enter the handler and

in here the TIMEUP flag is set. The process continues for each loop.

The following table is the pseudocode of the heartbeat monitoring program.

```
Start while loop:
  initialize H
  create socket
    if (TIME UP flag is set) ;     ping node
        if (node echo reply) ; status = OK
        else        status = FAIL;     initiate recovery
    else if (TIME UP flag is clear)
      set H to alarm timer
    create child process; accept socket connection with node
    just wait for heartbeat
      if (correct heartbeat message arrived)
          send receive signal to parent process
      else notify that wrong message is received
End while loop
Signal handler for heartbeat receive:
    unset timer to stop alarm
    update sampling and the next expected value for H
    terminate child process
Signal handler for timer elapsed for heartbeat expectation:
    unset timer to stop alarm
    set TIME UP flag
    terminate child process
```

Table 3.2: Algorithm for Heartbeat Monitor

HBM program was developed on C programming language using a number of POSIX and C standard libraries and socket communication. The fundamental of the HBM program is the timer that will set the estimated arrival time of the heartbeat message. This value is used to calculate the next heartbeat arrival as depicted in equation 2.8. Inside the program an alarm timer is set with

this calculated value. A heartbeat arrive will unset the timer. Otherwise if the heart beat does not come in time, the alarm will cause a signal interrupt to the program.

If the heart beat arrives in time, the child process will give signal to parent. Parent then terminates the child process and uses the time taken for each heart beat arrival to predict the next heart beat. This process continues until the timer elapsed during the waiting for the heart beat. In this case, the heart beat is considered missing with the possibility of a site crash, CPU getting slower or network instability. With benefit of doubt, the monitor pings the server to confirm its status.

If the server replies, the monitor acknowledges that server is alive and restart monitoring with the initial value of the threshold. Otherwise if the server is unreachable, it will declare that the server has failed and subsequently invoke the index server to start recovery steps via SSH command. The program consequently wait for the index server to perform its task of recovery of creating a virtual IP on a neighboring node which takes over the process of the failed server. This process is complete after neighbor selection is performed at IS and IS sends SSH command to selected fail-over neighbor which will turn on virtual IP on its $eth0$ network interface card.

The threshold is the expected heart beat arrival schedule 2.12.1. It is a

dynamic evaluation of the previous heart beat arrival from the beginning to the most recent. Heart beats sent over the network sometimes are affected by network bandwith and load. Therefore, it is necessary to consider network delay in this work. In the beginning the server initialization will take some delay that will gradually reduce with some minor irregularities. Over time the prediction value will closely assimilate the server and network current states. Any changes of the states can be detected promptly by the HBM program.

3.5.2 Confirming Failure

Under some circumstances the node fails to send a heart beat or a heart beat simply cannot reach the monitor in time even when the node is running like usual. This could be due to CPU loads or network latency. In order to be precise and not draw a false presumption about the server, the monitor program will utilize ping command to determine the status of the missing heart beat node. If a reply is received, the node is no longer in suspicion and the monitor program will reset the threshold to its initial value. It is necessary to reset the threshold and restart the monitoring process to draw a new assimilation of the network and server state as previous assimilation has been interrupted and is no longer relevant for the new state. On the other hand, if there is no reply and the ping utility concludes that the host is unreachable, the monitor program will declare the state of failure for the node and will

enter a recovery mode.

3.5.3 Selecting for Recovery

After a failure is confirmed, HBM sends SSH command to IS to start recovery process. The IS starts by gathering the failed node's information from its log. These information will later be sent to the fail-over node. Then, it finds the failed node's neighbors and begins checking their credibility to select the best replica for fail-over. All these processes involve checking its log and calculating the preset parameters of the replicas and their sites.

After the best fail-over is chosen, IS updates its log and sends SSH command to the selected node to become the fail-over for the failed node.

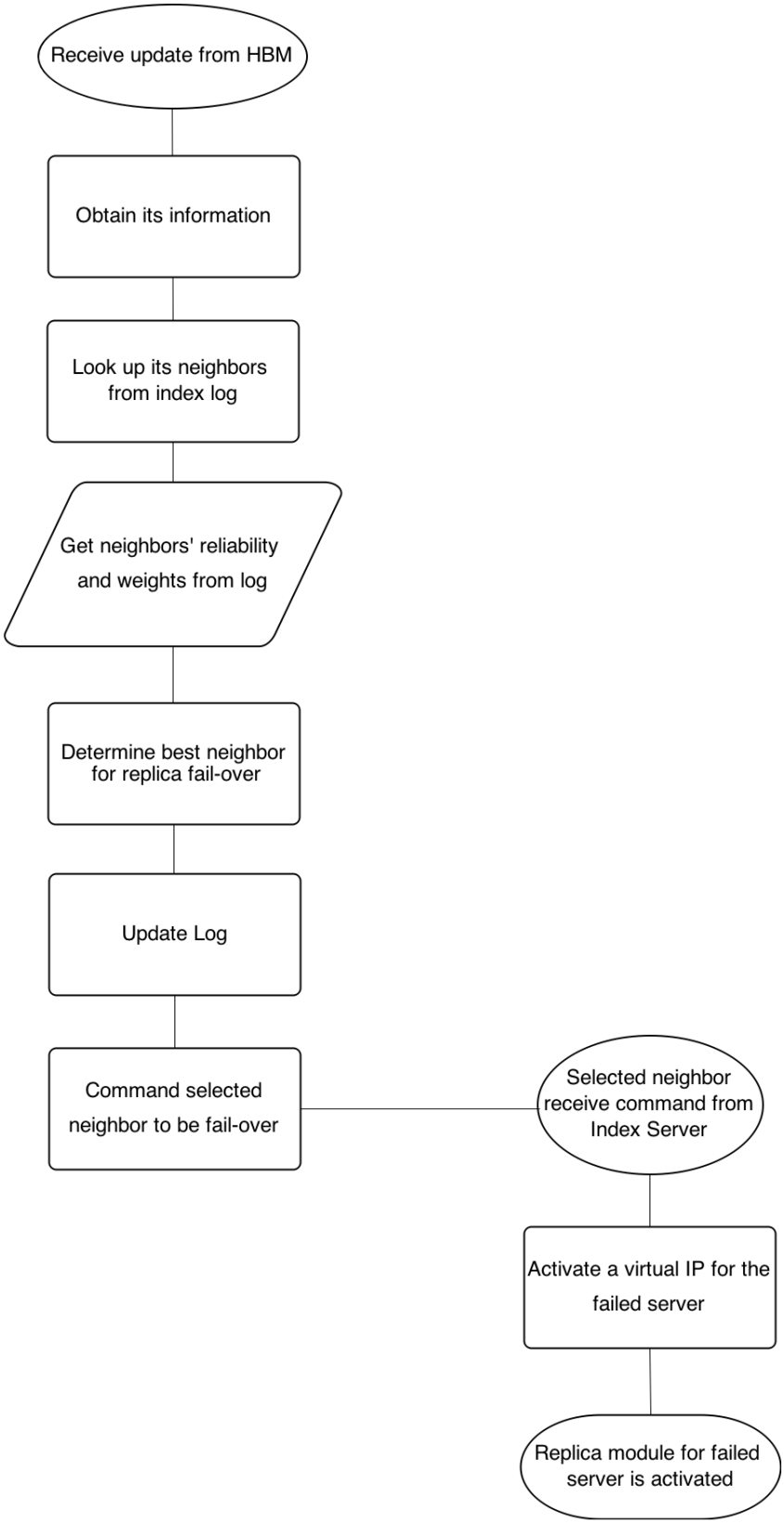

Figure 3.7: Flowchart of Failure Recovery Process

The flowchart above describes the process of Failure Recovery occuring at Index Server. The process is started after HBM declares the recovery mode. First of all the failed node's information is obtained from the log. Secondly, the program determines who are the nodes' neighbors. Subsequently, the neighbors are compared to determine who could be the best fail-over based on its preset scores in reliability and server condition. Lastly the program sends a command to the selected neighbor informing it to take over the service of the failed nodes. The node's information such as its IP address and netmask are also sent over. Receiving the command to be fail-over from IS, the node creates a virtual IP of the failed server IP address. Effectively the replica module of the failed server is activated and is served from the neighbor instead. The IS logs and the full algorithm of performrecovery are included in Appendix A and B.

3.6 Conclusion

This chapter has described a method to increase the availability of a database system by using several techniques. In the first phase, the system has to be modularized so that it is possible to distribute the system onto multiple separate sites based on different modules. This step creates a distributed environment where it prepares a policy for replication. The NRDT technique is chosen in which the nodes are logically arranged in a grid structure where

a node is neighbors with the adjacent nodes. This results in a 2 x 2 logical structure. This replication technique is adopted because it gives higher availability at low recovery latency compared to TRDT technique. For database system, it is proposed to use a certain arrangement involving master-slave relationship among several local databases and a key database. In the NABTICS new version, the remote server is used for any write operation to database while local server is used for reads. Finally all this setup for a distributed system with neighbor replication and database replication is complemented with a proxy for encapsulation to the outside world. Thereby a server cluster has been proposed for implementation.

A high availability framework is proposed which is featured with site monitoring, indexing, fault detection and autonomous recovery capabilities. This framework is consisted of two servers. The first server, Heartbeat Monitor (HBM) is responsible to monitor the live heartbeats of the nodes inside the cluster while the second server, Index Server (IS) is responsible for keeping logs and preparing for recovery. The heartbeat policy is proposed for monitoring the nodes at real time. As NABTICS is distributed into separate applications on different servers, this feature of Affirmative Adaptive Failure Detection is essential to ensure a contingency recovery if any hardware component is failed. A problematic server can be reported based on the outcome of the monitor program. Increased number in lost heart beats could indicate an

instability problem on the server. The monitor program takes affirmative action to ping the server to determine if its actually is lost on the network. This step ensures the fault detection accuracy. After all, false detection could cause unnecessary recovery which would waste resources and put dispensable load on recovery neighbor server. Furthermore, the fault detection technique is also prompt due to its close assimilation of the current server and network condition.

Once failure is confirmed, SSH command is sent to start recovery process on IS. The best node for fail-over is chosen based on specified weights and neighbor relationship. After the SSH command is sent to selected neighbor, Virtual IP is activated and on user site the service resumes automatically. Furthermore, data consistency can be ensured and available since database replication gives redundancy that eliminates SPOF and master control of data that can avoid conflicts in data operation.

Chapter 4

IMPLEMENTATION

This chapter describes the implementation of the discussion in methodology to demonstrate the new framework for NABTICS application. The implementation uses the standard LAMP (Linux, Apache, MySQL, PHP) stack for the web application side and socket and SSH communication for the middleware layer. In the result, it will show that the new implementation can improve the availability of NABTICS application.

4.1 Introduction

The implementation requires both hardware and software components. To demonstrate the system, the NABTICS application is broken down into four primary applications and spread across four sites. On each site, the replica of its immediate neighbor site's application is maintained using Neighbor Replication Technique(discussed in 2.15.2). These four server nodes make up a server cluster that connect on private internal network and to a fifth server namely the proxy. The proxy is connected to internet and is prepared with a web server. Client can access the four application servers through the proxy.

The statuses of the four application servers are observed and recorded in a log by a partnership of another two services namely Heartbeat Monitor (HBM) and Index Server (IS). This partnership is also responsible for performing a failure recovery process and choosing a fail-over server and command the replica to be activated.

4.2 Hardware Specification

For this experiment, a Windows 7 PC with Intel i3 and 8GB RAM is used to host the virtual servers for this experiment using a hypervisor, VMWare Player. VMWare provides an experimental testbed that allows users to configure VLAN-based private network and specify the operating systems on the virtual nodes. The VLAN-based network topology for this implementation is a single internal LAN connecting all the seven nodes and a NAT based network to connect one proxy server to the internet. The network topology for this work is summarized in following table.

Server Name	**Server type**	**VMWare Network Setting**	**IP Address**
HBM	Application server MySQL server	Host-only	10.0.0.10 (LAN)
IS	Application server	Host-only	10.0.0.20 (LAN)
Node1	Application server Web server MySQL server	Host-only	10.0.0.1 (LAN)
Node2	Application server Web server MySQL server	Host-only	10.0.0.2 (LAN)
Node3	Application server Web server MySQL server	Host-only	10.0.0.3 (LAN)
Node4	Application server Web server MySQL server	Host-only	10.0.0.4 (LAN)
Proxy	Proxy server Web server	Host-only NAT(second NIC)	10.0.0.123 (LAN) 192.168.226.150 (External)

Table 4.1: Server types and network settings

4.3 Software Specification

In this implementation, CentOS 5 is the operating system on all virtual nodes. Linux offers plethora of system resources which makes it a powerful platform for system development. Other than system resources, different tools were used for development, debugging, testing and execution. They are described in the following.

1. C Programming

   POSIX and standard C libraries were used for string manipulation, real time, socket communication and signals. $fork()$ system call was used to create concurrent processes inside heart beat monitoring program.

2. GDB

   GNU Debugger or GDB is the standard debugger for C programs on Linux platform. It offers extensive facilities for tracing and altering the execution of computer programs.

3. GCC compiler

   GNU C Compiler or GCC was used to compile and link programs with its libraries to build complete executables.

4. BASH programming

   Bash is a command processor that typically runs in a text window, where the user types commands that cause actions. Bash can also read commands from a file, called a script. In this research, commands lines and scriptings were used in on all nodes for some purposes. Specifically for Index server, the programs were prepared in BASH script. For running the program, it is called in BASH on Linux terminal.

5. Ping utility

   Ping utility is a convenient tool provided by most Operating Systems to test reachability of a host on an IP protocol. It is used throughout the development and initial set up to confirm network connection and during runtime it is used to confirm the state of a node after it is suspected to have failure.

6. SSH

   SSH or Secure Shell is a UNIX-based command interface and protocol for secure access to remote computer. It uses RSA public key cryptography for both connection and authentication. In this research, SSH is used for remote command and also FTP file transfer. For remote invocation in fault recovery steps, the SSH connection is made passwordless to automate the process.

7. MySQL Master-Slave replication

   MySQL Master-Slave replication scheme enables one server to have multiple replicas on remote computers. It is especially convenient to back up data and build data distribution on multiple sites. In MySQL replication is asynchronous meaning slave databases do not need to be connected to the master database all the time [31]. Special settings are required to keep the slaves database (replicas) in synch with the master database.

8. Apache

   Apache is a web server application that is most widely used in the world. In this research apache is also used as forward and reverse proxy for cluster nodes and internet communication.

On client-end, the application is accessed through a proxy server. The proxy server is connected to the cluster nodes on a private internal network. Outside servers cannot communicate directly with the application nodes. This way the architecture of the distributed application delivers transparency to clients where clients only query on a single IP despite that the applications come from different sources.

4.4 Development

Software development was done for these programs.

1. NABTICS application, using PHP, MySQL and HTML
2. Heartbeat Monitor, using C programming
3. Heartbeat Generator, using C programming
4. Failure Recovery, using Bourne Shell (BASH) programming

4.5 Server Cluster

Having a cluster, the application nodes and database can sit behind a firewall and are more secure against threats and attacks. Besides this, on the system level, the proxy also enables the cluster nodes to access internet for system updates through yum tools where the proxy is specified in yum configuration and not through Apache setting. This is called a forward proxy.

While proxy provides service for requests, all processing are done inside the cluster. For reads operation, it will access local data whereas for data updates it will access a remote MySQL server that is on the same private network.

4.6 Setting up for running

Before running, all the seven servers must be set up for their specific purposes. The setup is necessary for it to perform its specific function and to communicate with other nodes. The setup is described below for each item.

All MySQL servers are required to have port 3306 opened. On CentOS 5, this configuration is at /etc/sysconfig/iptables.

First of all, on all the servers the host tables (/etc/hosts) are configured to register all nodes' addresses locally. It is necessary so that each node know each other by name. Retyping the IP address is not necessary and can keep codes and command lines more manageable.

10.0.0.10	hbmonitor
10.0.0.20	indexserver
10.0.0.1	node1
10.0.0.2	node2
10.0.0.3	node3
10.0.0.4	node4
10.0.0.123	proxy

Table 4.2: Host table

The four application nodes and proxy server are configured to be web servers using Apache. The appropriate folder for DocumentRoot are set. It is also set to allow all to access the folder.

4.7 Setting up proxy

In this research, Apache proxy is used for its popularity and versatility and it is a component of the LAMP stack. It is often used in high-load environments. But for this implementation, mod_proxy simply redirects requests for certain URLs to another web server in this research the application nodes, so it requires no additional configuration on the application server other than for reverse proxy. After apache has been updated using yum command, the httpd.conf file is configured as below in summary:

1. The proxy module is enabled by using *LoadModule*
2. *ProxyRequests* is set to ON
3. Using *ProxyPass* and *ProxyPassReverse*, the appropriate aliases are set for the applications location to include them in NABTICS homepage links like in figure 4.1.

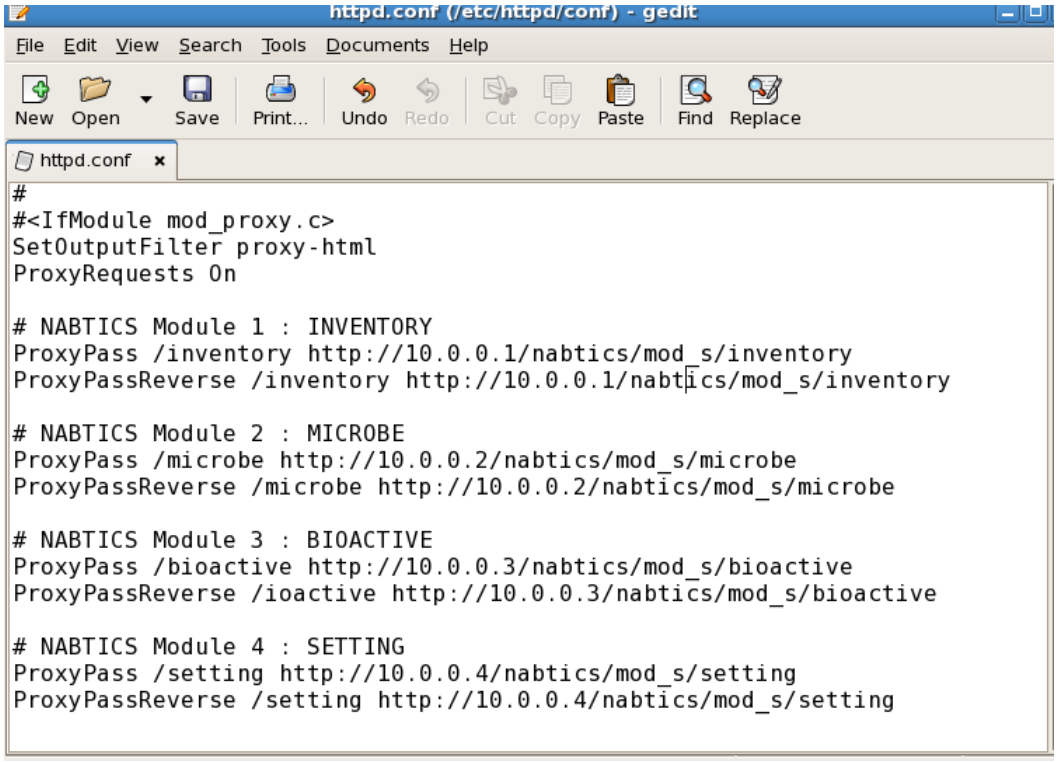

Figure 4.1: NABTICS modules setting in Proxy server

4.8 NABTICS System

There are several changes that need to be done on the NABTICS system. The original homepage is consisted of four modules and personalized view for user. For simplicity in this work, the first level user authentication is disabled.

On the homepage, the links to the modules are modified to contain just the aliases of the modules as shown in the following example. The proxy reverses to the specific locations at other nodes perfectly hiding the meaning

of the aliases to users.

```
<a href="/setting" target="main">Setting</a><br>
<a href="/bioactive" target="main">Bioactive</a><br>
<a href="/microbe" target="main">Microbe</a><br>
<a href="/inventory" target="main">Inventory</a><br
```

Figure 4.2: NABTICS Homepage's Links

Depending on the structure of the system, the modularization process although very straight-forward, it can take significant work to identify and manage the common points inside the modules. As the modules are to be independent of each other, the common points need to be localized.

In the basic prototype of the new version of NABTICS, minimum localization is done. The fundamental things are the use of two different databases for write and read as opposed to common database in original version. Therefore, two connections are maintained throughout the modules and are switched for the different operations. Next, the appropriate foldering needs to be set up for each module. Inside the folder, all necessary files are included and are configured to switch off for other modules. This is to avoid error at the individualized module. Subsequently, these modules will be copied accordingly onto different nodes to set up neighbor replication. This is described in table 3.1.

The homepage of the NABTICS modules is placed at the proxy server while all other modules are placed on the application nodes with each a local database.

4.9 Setting up MySQL for Database Replication

For MySQL communication all servers must open port 3306. The following steps are important to ensure data integrity in NABTICS through having database replication. In this set up, the four application nodes will have each a local database which is a slave to the master database hosted on Heartbeat Monitor.

1. Setting up master database

i. Ensure that port 3306 is allowed in iptables setting (/etc/sysconfig/iptables). This step was already taken in server set up.

ii. Insert the following configuration in MySQL (/etc/my.cnf)

my.cnf

```
[mysqld]
server-id = 1
binlog-do-db=nabtics
expire-logs-days=7
relay-log = /var/lib/mysql/mysql-relay-bin
relay-log-index = /var/lib/mysql/mysql-relay-bin.index
log-error = /var/lib/mysql/mysql.err
master-info-file = /var/lib/mysql/mysql-master.info
relay-log-info-file = /var/lib/mysql/mysql-relay-log.info
log-bin = mysql-bin
```

iii. Here 'nabtics' is the database name to be replicated to the slaves.

iv. Restart MySQL service by running $ *service mysqld restart*

v. Login to MySQL to create a slave username and password for all the member nodes' local databases. Here is an example for node 1 with IP address 10.0.0.10

```
mysql> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> GRANT REPLICATION SLAVE ON *.* TO 'root'@'10.0.0.10' IDENTIFIED BY 'microbe';
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH TABLES WITH READ LOCK;
Query OK, 0 rows affected (0.00 sec)
```

vi. Restart MySQL service by running $ *service mysqld restart*

vii. Login to MySQL and enter command SHOW MASTER STATUS to show master log and position

```
mysql> SHOW MASTER STATUS;
+------------------+----------+--------------+------------------+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+------------------+----------+--------------+------------------+
| mysql-bin.000001 |      106 | nabtics      |                  |
+------------------+----------+--------------+------------------+
1 row in set (0.01 sec)
```

2. Setting up slave database

i. Ensure that port 3306 is allowed in iptables setting (/etc/sysconfig/iptables).

This step was already taken in server set up.

ii. Insert the following configuration in MySQL (/etc/my.cnf)

```
my.cnf ×
server-id=5
master-host=10.0.0.10
master-connect-retry=60
master-user=root
master-password=microbe
replicate-do-db=nabtics
binlog-do-db=nabtics
relay-log = /var/lib/mysql/mysql-relay-bin
relay-log-index = /var/lib/mysql/mysql-relay-bin.index
log-error = /var/lib/mysql/mysql.err
master-info-file = /var/lib/mysql/mysql-master.info
relay-log-info-file = /var/lib/mysql/mysql-relay-log.info
log-bin = /var/lib/mysql/mysql-bin
```

iii. Restart MySQL service by running $ *service mysqld restart*

iv. Now log in to MySQL as root user and tell the slave server to where to look for Master log file which has been created on Master server in previous steps

```
mysql> SLAVE STOP;
Query OK, 0 rows affected (0.01 sec)

mysql> CHANGE MASTER TO MASTER_HOST='10.0.0.10', MASTER_USER='root',
 MASTER_PASSWORD='microbe', MASTER_LOG_FILE='mysql-bin.000001', MASTER_LOG_POS=106;
Query OK, 0 rows affected (0.03 sec)

mysql> SLAVE START;
Query OK, 0 rows affected (0.01 sec)
```

v. Run SHOW SLAVE STATUS/g to see slave status and details

4.10 Setting up SSH for secure remote command

The following describes how to set up passwordless SSH for secure remote command between HBM to IS, and IS to all node servers. The term passwordless SSH or OpenSSH means that SSH authentication is carried out by using public and private keys. Using public/private key authentication with SSH enables SSH logins without requiring passwords interactively and this is known as SSH key-authentication.

Because passwordless SSH or OpenSSH allows a system to run commands on remote systems, showing the results directly, as well as just logging in to systems, it is ideal for automating common tasks with shell scripts. An important step in configuring fault recovery here is enabling the Index Server to be able to login into all of the member nodes without any authentication.

To do this, Index Server must be able to SSH into all member nodes without password authentication. So that programs reside in member sites can be triggered automatically. The administrator needs to create the key on index server and copy it to the other nodes.

Before creating the login keys for the nodes, administrator will also need

to make sure that all of nodes have the exact same user account name. Here is how to configure SSH to connect to any member site without a password.

For this setting example local server is IS server while remote server is node1.

i. Generate DSA key pair

Use ssh-keygen command as follows:

$ ssh-keygen -t rsa

Output:

```
Generating public/private rsa key pair.

Enter file in which to save the key (/root/.ssh/id_rsa):

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in (/root/.ssh/id_rsa).

Your public key has been saved in (/root/.ssh/id_rsa.pub).

The key fingerprint is:

ac:a7:78:07:91:41:75:a5:1e:b1:96:1f:7c:1a:a3:dc
```

The command *$ssh-keygen -t rsa* initiated the creation of the key pair. No paraphrase was entered (Enter key was pressed instead). The private key was saved in /home/root/.ssh/id_dsa. This file is read-only and only for the user. No one else must see the content of that file, as it is used to

decrypt all correspondence encrypted with the public key. The public key is saved in /home/root/.ssh/id_dsa.pub

ii. Set directory permission

Next, make sure a user have the correct permission on .ssh directory:

$ cd

$ chmod 755 .ssh

iii. Login to remote server (node1) and make sure permissions are set correct:

$ chmod 600 /.ssh/authorized_keys

iv. Copy the file to node1 from Index Server using the following:

$ scp /.ssh/id_rsa.pub root@node1:.ssh/authorized_keys

For the rest of the member nodes the same authentication key of Index Server must be used.

Setting up SSH with RSA public key authentication (passwordless login) from HBM to IS server will use the same procedures. An authentication_key must first be created at HBM.

4.11 Fault Detection Configuration

To ensure failure detection to function on a single command line, the steps below are very important.

i. Installing heart beat generator on each node. For this, the port used for sending out heartbeat must be the same as the port selected to receive heartbeat on the HBM side. This can be specified at the 'port' variable inside the C script. If the script has been changed it needs to be rebuild again into executable file. This can be done by command line gcc.

gcc –hbgen.c –o hbgen

ii. Installing the heart beat monitoring program at HBM. The same prerequisite is the same as in step 1 except HBM program needs to be linked with real-time library. The command line to rebuild the HBM program is.

gcc –W –hbm.c –o hbm –lrt

iii. Setting passwordless SSH login for HBM on IS. This step can be referred in previous section.

Instead of a command line to execute the heartbeat monitoring routine, daemon configuration is also possible to let the monitoring and heartbeat generator program run in the background without user participation. With autonomous approach, the heart beat monitoring program can run at system boot. This way admin does not need to start heartbeat generator and monitor manually. In fact, most daemons are ordinarily started at system boot and runs until system shut down, unless it is forcibly terminated. For testing

and development purposes, the heart beat programs in this research are not configured into daemons for experiment simplification.

4.12 Failure Recovery Configuration

To ensure the proposed fault tolerant mechanism can do self-recovery during the event of fail-stop failure, the following configuration is necessary to prepare for fault recovery system.

i. Setup SSH with RSA public key authentication for IS server login to all application nodes

   This step was discussed in previous section.

ii. Install recovery script at IS.

   It is necessary to enter this command before executing the script for the first time.

   $ chmod +x performrecovery.sh

4.13 Executing the fault tolerant framework

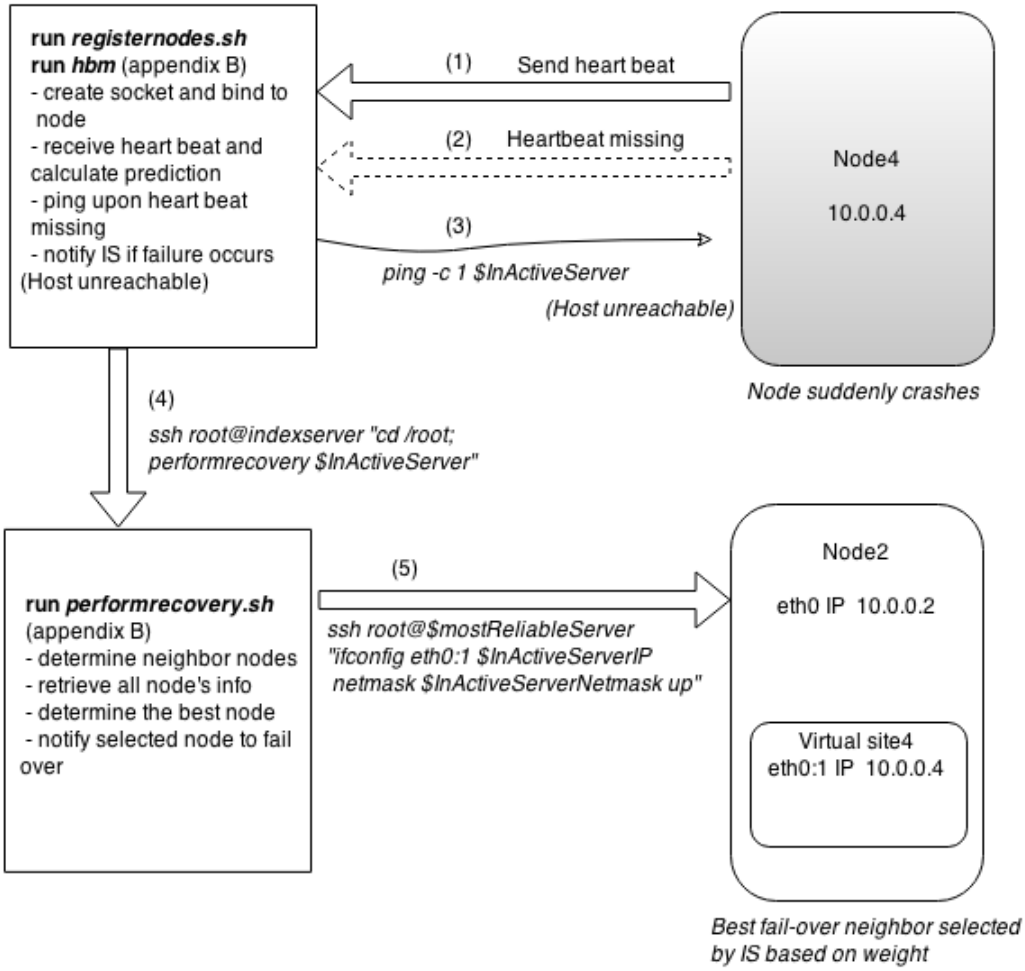

Figure 4.3: The fault-tolerant framework

The working of the fault-tolerant framework is illustrated in the diagram. In this diagram, the fault tolerant steps are demonstrated in sequence. The terminal output of figure 4-2 are described in section 5.4.2.

Based on the working flow described in previous illustration, the following

are required procedures to run the fault tolerant system.

i. Register all member nodes by running ‘registernodes.sh’ executable on command line at HBM. This will transfer tables of nodes’ informations to Index Server through FTP transfer. The tables are described in Appendix.

ii. Execute ’hbm node4’ at HBM terminal to monitor for node4 for example. This will start heartbeat receiving and prediction.

iii. Execute ’hbgen’ executable at application nodes terminal. This will start periodic heartbeat to be sent over to HBM to indicate node’s aliveness.

4.14 Recovery process

It can be seen that over time, the expectation of next heartbeat arrival time gradually reduces from initial value of 8 ms and become more closer to receive time, which is the period of the heartbeat frequency from the application node as arrived on HBM port. When a failure has been detected, the node is in state of suspicion because its actual state is not yet confirmed. The state of suspicion is raised in two situations:

i. Failing to bind socket with the node being monitored during runtime. This could be due to sudden node crash, socket reject or hbgen program

malfunction at the node side.

ii. Not receiving heartbeat within the expected time even when socket is connected. This raises the state of timeout.

These two situations will trigger the HBM program to check on the node by sending a "ping echo request" to the node. If the result is "Destination host is unreachable", the node is confirmed to have failed. HBM will enter recovery mode by sending a SSH command to IS to start recovery process. Otherwise, if ping reply is received, HBM program will erase the state of suspicion and continue monitoring the node like usual. At this point the expected time threshold has been reset to initial value of 8 ms. After ping echo request fails to get reply from node, HBM declares that the node has failed and instigate IS to do recovery. The information sent over to IS is the node that has failed then IS will start the process of recovery by first identifying the neighbors of the node and making a selection on the best neighbor to be the fail-over server based on a certain criteria. After the neighbor has been selected, IS will send an SSH command to it to be the fail-over server of the recently failed node by activating a virtual IP address to relocate the failed node. As a result, the IP address that originally directs to the application node that has failed now is directing to other fail-over node. The application is resumed on other site without significant downtime. User does experience continuity of service.

4.15 Conclusion

For this research, a server cluster was created to group the application nodes of NABTICS and placed behind a proxy for security. to complete the nabtics system, each primary application is resided at a dedicated server. On each of these nodes, the copy of its immediate neighbor is maintained as described in Figure 4.2. these nodes are prepared to be fail-over for their neighbors (after a selection process has determined the best fail-over node). It is illustrated in Figure 4.3. The proxy server has connection to the internet and is a gateway for the distributed application as seen in Figure 4.1. It is a web server that forwards request and service from others. It hosts the main page of NABTICS consisting links to its four primary applications as seen in Figure 4.2. The clients would request for these URLs and the proxy returns them to clients with transparency of the resources. In this way, a single IP can be used for NABTICS. All of this setting, delivers transparency to clients both in the events of failure and non-failure.

Chapter 5

RESULTS AND ANALYSIS

In this chapter, the project is tested to present qualitative and quantitative results that will be the basis for review and discussions.

5.1 Introduction

In order to evaluate the proposed methodology, the experiments are conducted within the environment setting as demonstrated in chapter 4. The analyses are started with failure detection, followed by failure recovery and lastly the final framework where the availability improvements of the NABTICS modules are discussed and the process from failure to recovery is observed.

5.2 Failure detection analysis

In heartbeat detection, the nodes send a periodical heartbeat messages and are timely received by Heart beat monitor via a committed C socket. The following graphs records the heartbeat sending and receiving in a sequential timeframe.

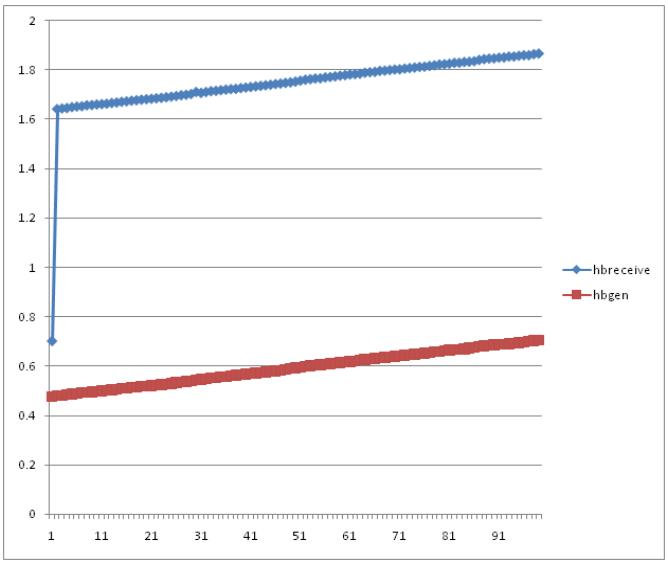

Figure 5.1: Heartbeat sending and receiving

From the graph, it is apparent that HBM receives heartbeat time is directly proportional to HB generator heartbeat sending time. HBM receives heartbeat from node with a transient delay in an adequately consistent manner under a reliable network. In each heartbeat cycle the heartbeat generator program in node tries to connect to HBM socket. The HBM program responds and accepts the connection otherwise the HB generator program will halt itself. This ensures all heartbeat are sent to a discreet direction. This setting delivers consistency in transaction of heartbeat as such depicted in above graphs. In the case where HBM commits to socket but heartbeat is not received within expected time, HBM will suspect a failure and take precautionary steps to confirm. On the other hand, when HBM fails to accepts a socket connection for three consecutive times when initially it was able to connect, the alert for

failure is also raised.

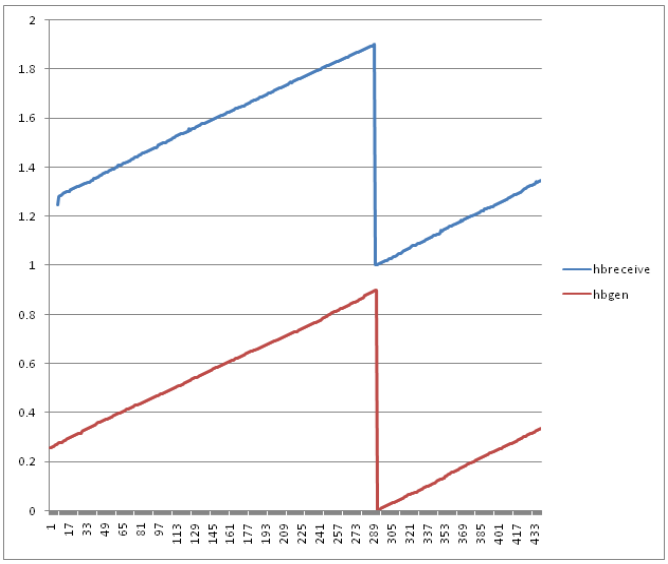

Figure 5.2: Heartbeat sending and receiving for longer sample

In the graph, the heartbeat generation time increments for each cycle due to CPU loading on the application node. Over time, the heartbeat generation time takes a different patterns as can be seen in Figure 5-2 as the C looping process in the heartbeat generation is influenced by CPU load.

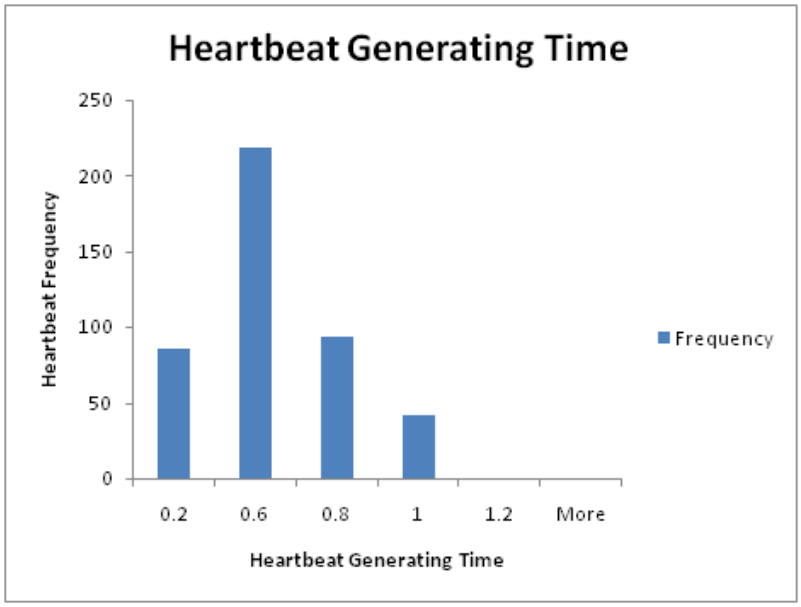

Figure 5.3: Distribution of heartbeat generation time

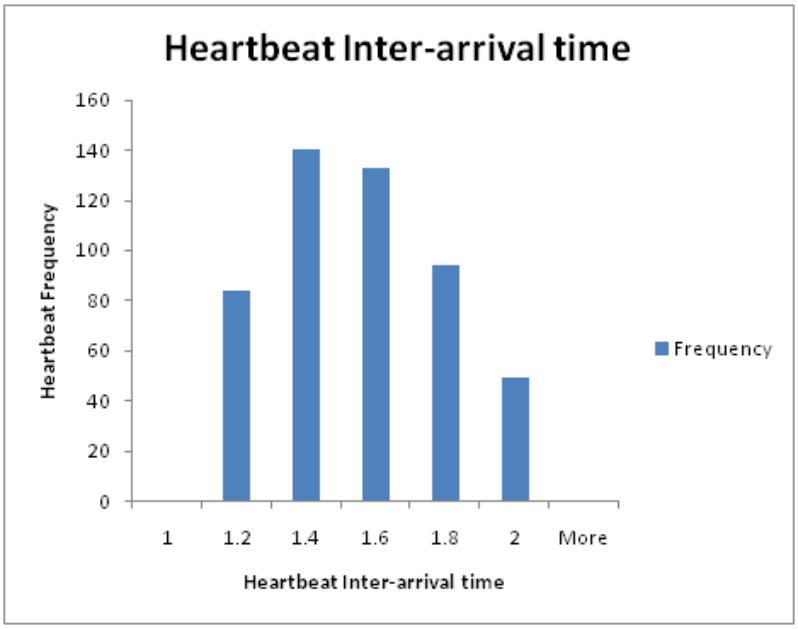

Figure 5.4: Distribution of heartbeat receiving time

Figure 5.3 and 5.4 give the results in histogram, representing the frequency

of heart beat generation and receiving from the application nodes to the HBM. This comparison is to observe the similar patterns obtained for the data of both implementations. It is also observed that the data are irregular when network and CPU load change. The best detection technique must adapt to the margin in order to optimize the detection time with accuracy and completeness [20].

As heartbeat generation time is not consistent and directly affected by CPU load condition, the detection for failure must acclimate to this changes. The expected time threshold 2.12.1 gives timely detection with accuracy and completeness. Because change is constant, a gradual change is normal and expected but an abrupt change could signal a problem. The following graph in Figure 5-5 shows the threshold value gradually adapts to heartbeat reception time. As a conclusion from this graph, the threshold gives a good representation for the network or CPU condition at any instance. Therefore, the ETA value is the key to catch if the monitored node make an abrupt change which could mean a problem.

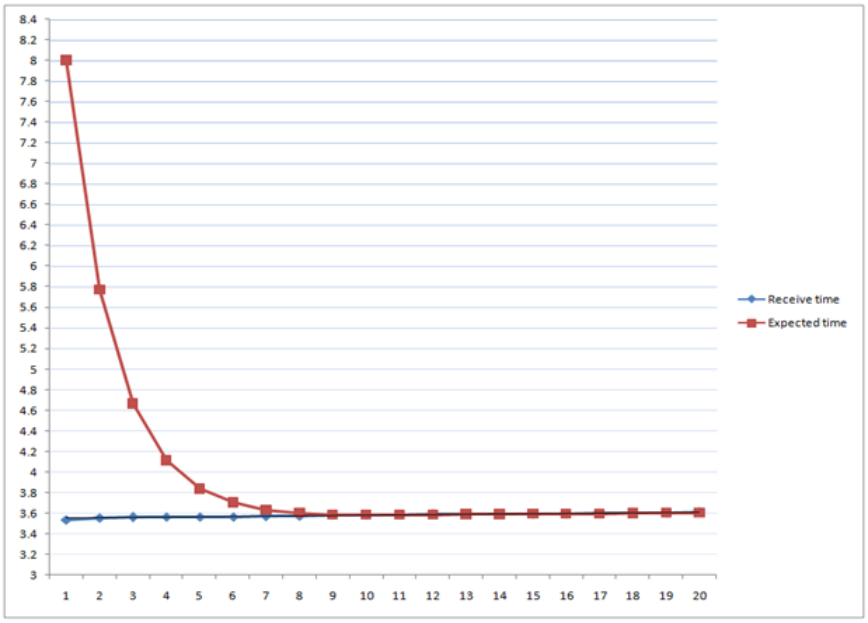

Figure 5.5: Expected time of arrival adapting to heartbeat inter-arrival time

In this project, the methodology is using equation 2.8 in chapter two for the Affirmative Adaptive Failure Detection. It is proven better than several other techniques [20] for the following reasons:

i. It scored among top in prompt detection metric, without compensating accuracy. In an experiment for comparison between techniques, it is the second fastest technique with the least number of false detection. There are also other techniques that give high accuracy and complete detection but perform poorly at prompt detection. In the experiments conducted, the AAFD technique outperforms other techniques by an average of 30%.

ii. Using ping utility, the technique is affirmative and can avoid false detection

iii. It takes the least time to declare a permanent site failure.

iv. The technique does not overestimate or underestimate the actual inter-arrival HB time.

In this proposed work, the failure detection is designed to respond to three failures causes. They are server total fault in which case the server is completely failed, network cut or instability which could be temporary or permanent or port disable (heartbeat generator malfunction).

The monitor detects failure if socket *accept* returns fail for three consecutive times without having to confirm on ping echo reply. This is because server is still alive only not able to send heartbeat that could be due to port malfunction or heartbeat generator program hang/terminated. This is also a state of malfunction since no heart beat means monitoring cannot be performed. But it may not be necessary to invoke a fail-over recovery because server may still be alive.

The failure causes were simulated to observe the results. In the first fault test, the node was stopped by pausing the VMWare player. In the second test, the network card on monitored node was shut down using terminal command line. In the last test, the heartbeat generator program was terminated during execution. All these fault simulations are detected promptly by the HBM

program.

5.3 Failure recovery analysis

Depending on replication architecture and policy in providing availability of components, different techniques have different associated cost of managing replicated data [6]. In this section, the focus will be on reinstating the environment based on Neighbor Replication technique. The recovery time in this implementation is divided into three parts.

i. The time to send notification of a failure site from HBM to IS

ii. The time taken for the system to search for the nodes that hold the same service as the failed node, then for nodes verification and neighbor selection.

iii. The time taken to send notification to selected neighbor node and virtual IP activation

From theory the time to send notification of a failure site from HBM to IS and the time taken to send signal for virtual IP activation at selected neighbor nodes are constant and not affected by the number of replicas. This is because it does not require any processing. Recovery time depends on replication

technique. This is affected by the different amount of replicated nodes.

In [20], the experiment to compare recovery latency for different replication techniques was conducted. It is found that, for some techniques they can give a fast recovery at a certain n nodes, however the latency increases as the number of nodes, n increases in the environment. This is because for these techniques, the number of replicas increase with more nodes in the environment. For another technique that employs Two-Replication Technique called TRDT (described in section 2.15.1), it gives the fastest recovery and its latency is not affected by the number of nodes in the environment. But its drawback is that in every fail-over, it does not have any more copy for future recovery. Keeping only one copy can jeopardize the availability of the module or data. From the discussion, it is estimated that the recovery latency is directly proportional to the number of replicas for a module or data item.

To design a distributed system with multiple sites in joint parallel-serial logical structure, the neighbor replication technique is better because the complexity of replication does not increase with the number of nodes, if good recovery latency and availability are the desired attributes. The neighbor replication technique used for NABTICS gives a recovery time that is not affected proportionally by the number of nodes while still gives a improved availability for each module even if more new modules are introduced into the system in the future.

The NABTICS cluster can withstand two consecutive permanent server failures for all modules to be functioning. After the first permanent crash, the system is now down to less redundancy as some replicas are eliminated. If a second server is to crash, two modules will be left with only one replica each while the other two modules have two replicas each. The two modules with one replica do not have any more backup. With that said, the NABTICS cluster can fully function with at least any two servers in the cluster. On the other hand, if only one server is alive in the cluster, only 3 modules will be available out of 4 modules of NABTICS in this implementation.

5.4 System Performance

In this section, the performance of the new version of NABTICS will be discussed. After every implementation procedure in chapter 4 is completed, the system is ready to perform under the new environment.

A single point of failure (SPOF) exists when a hardware or software component of a system can potentially bring down the entire system without any means of quick recovery. Highly available systems tend to avoid a single point of failure by using redundancy in every operation. Adding redundancy results in more parallelness in the system. Discussion on system availability in serial and parallel architecture is done in section 2.8.

5.4.1 The serial and parallel model of the system

Figure below describes the serial and parallel components in the new distributed NABTICS where inside each box represents parallelness of the modules copies and the bigger squares represent the modules in serial with each other making up the NABTICS system.

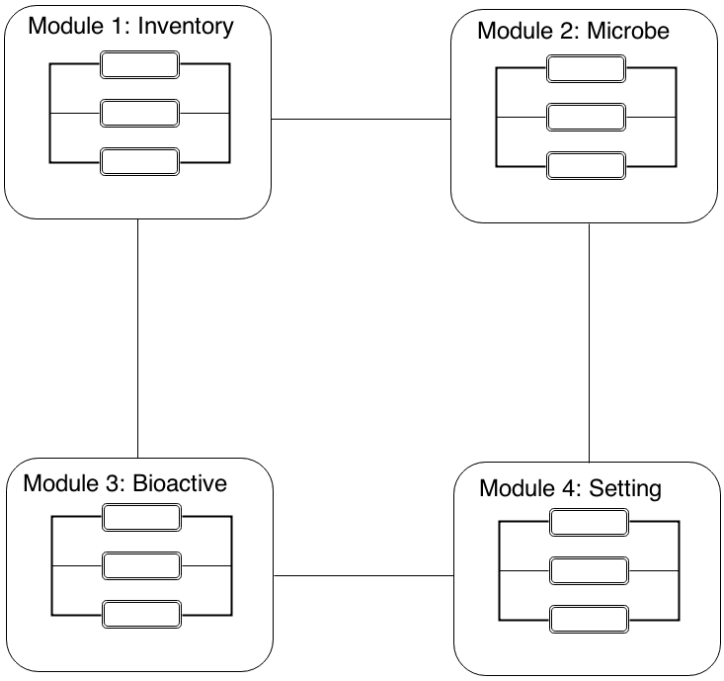

Figure 5.6: Joint serial and paralel model in distributed NABTICS

Each module has two replicas that reside in the neighboring servers. In other words, each module has three parallel components. Fixed from the original version is the serialness of the modules to one another. This can be explained in the sense that each module is unique and without one the whole

system cannot function. After adding redundancy, each module has increased in its availability since it is no longer a single point of failure in of itself. When availability is increased at module level, the total system availability is also increased as formulated in equation 2.4. Using equation 2.6, the availability increment for each module and the whole system are calculated.

The two assumptions made in this calculation are

i. Each server inside the cluster has predisposition level of availability as following:

   Node 1 : 90%, Node 2 : 80%, Node 3 : 90%, Node 4 : 90%

ii. Each module replica has the availability of the server it is on.

Without any replication, each module is only one copy and hosted at individual server. The modules availability levels are:

Availability of Module1($Inventory$) = 90%

Availability of Module2($Microbe$) = 80%

Availability of Module3($Bioactive$) = 90%

Availability of Module4($Setting$) = 90%

The availability of the whole system is then $90\% * 80\% * 90\% * 90\% = 58.3\%$

With neighbor replication, each module now has a copy at each of its immediate neighbors as can be seen in figure. Therefore each module now has new availability level as there are three replicas in parallel for each module.

Availability of Module1($Inventory$) = $1-(1-90\%)*(1-80\%)*(1-90\%) = 99.8\%$

Availability of Module2($Microbe$) = $1-(1-80\%)*(1-90\%)*(1-90\%) = 99.8\%$

Availability of Module3($Bioactive$) = $1-(1-90\%)*(1-90\%)*(1-80\%) = 99.9\%$

Availability of Module4($Setting$) = $1-(1-90\%)*(1-90\%)*(1-80\%) = 99.8\%$

The availability of the whole system is then $99.8\%*99.8\%*99.9\%*99.8\% = 99.3\%$.

So the availability of the NABTICS system has increased from 58.3% to 99.3%. The availability is improved by 70% using the neighbor replication technique.

The more number of parallel replicas will yield more availability. However, adding more servers for replication can incur extra cost therefore the most cost-efficient way is to have as much replicas by utilizing available resources

within the environment. In determining the replication technique to deploy, the system designer must consider how much recovery latency the system can tolerate without hurting user experience and must consider if the system can use the minimum parallelity it needs to achieve high availability. Direct correlation between number of replicas to number of nodes will cause slower recovery if there are more sites in the environment. However, using the neighbor replication technique, this problem can be avoided as complexity of replication does not increase with the number of nodes. Since only immediate neighbors are candidates for fail-over, the processing is quite the same for more number of nodes.

5.4.2 From Failure to Recovery

Recovery taking place after a failure is confirmed, is a process to return to a normal state. In this experiment, recovery is reinstating the service continuity and user experience autonomously and directly boosts availability to the system. The HBM which is responsible to monitor nodes' liveness, will also initiate confirmation of failure suspicion and recovery process. The IS will prepare for recovery by selecting a fail-over and informing the neighbor node. The failed server's task will be assumed by its selected neighbor. These steps are observed inside HBM terminal during failure detection until recovery.

In this experiment, module4 is opened on browser through the homepage of NABTICS that is hosted on the proxy server. After some time the server node4 will be shut down.

```
root@hbmonitor:~
File  Edit  View  Terminal  Tabs  Help
==============

  Ack Hbeat node4 Rtime=1.815517 ms, Exptime=1.907983 ms
==============

  Ack Hbeat node4 Rtime=1.816219 ms, Exptime=1.861750 ms
==============

  Ack Hbeat node4 Rtime=1.818439 ms, Exptime=1.838984 ms
==============

  Ack Hbeat node4 Rtime=1.820357 ms, Exptime=1.828712 ms
==============

  Ack Hbeat node4 Rtime=1.821934 ms, Exptime=1.824534 ms
==============

  Ack Hbeat node4 Rtime=1.825098 ms, Exptime=1.823234 ms
==============

  Ack Hbeat node4 Rtime=1.828863 ms, Exptime=1.824166 ms
==============
```

Figure 5.7: Monitoring heartbeat from node4

While node4 is still alive, it sends heartbeat messages to HBM at a frequency period of approximately 1 ms. The heartbeat messages are timely received by HBM as can be seen in *Rtime* value. The value in *Exptime* is the expectation of the next *Rtime* maximum value.

During normal execution, timeout may occur due to abrupt change in network or CPU load of node4 causing a delay in sending heartbeat message as depicted in figure 5-8. After a timeout, HBM sends a ping echo request to

the node and waits for its reply. After receiving a ping reply, the HBM will resume monitoring with a reset value for *Exptime* and consider node4 status is alive.

```
root@hbmonitor:~
File  Edit  View  Terminal  Tabs  Help
  Ack Hbeat node4 Rtime=1.707684 ms, Exptime=1.703625 ms
==============

  Ack Hbeat node4 Rtime=1.708627 ms, Exptime=1.705655 ms
==============

  Ack Hbeat node4 Rtime=1.711213 ms, Exptime=1.707141 ms
==============

  Ack Hbeat node4 Rtime=1.712573 ms, Exptime=1.709177 ms
==============

  Ack Hbeat node4 Rtime=1.715420 ms, Exptime=1.710875 ms
==============

  Ack Hbeat node4 Rtime=1.716474 ms, Exptime=1.713148 ms
==============

 TIMEOUT 1.971436 while exptime 1.714811 ms
ping and waiting for response

=== Ack ping reply node4 ===     [OK]
```

Figure 5.8: A timeout occured but is not a node failure

If it is indeed a failure, where the ping echo request from HBM can not reach the host, HBM has confirmed the node failure. It then invokes IS to start the recovery process. As can be seen in figure 5-9, IS takes several steps before sending SSH command for fail-over to the selected neighbor of node4.

```
root@hbmonitor:~
File  Edit  View  Terminal  Tabs  Help
==============

  Ack Hbeat node4 Rtime=1.828863 ms, Exptime=1.824166 ms
==============

 TIMEOUT 1.982614 while exptime 1.826514 ms
ping and waiting for response

=== node4 current status       [FAILED]

Invoking IndexServer to perform recovery on node4

IndexServer:  Finding the neighbors of node4

IndexServer:  Determining the best neighbor

IndexServer:  Most reliable server is node2 for weighted = 16
IndexServer:  Time taken to find the most reliable node is 0.035 seconds

IndexServer:  Updating server status for node4

IndexServer:  Activating the node2 to be fail-over for 10.0.0.4..
```

Figure 5.9: Failure is detected and confirmed

In this experiment, node2 is chosen to be the fail-over for node4. This implies that the replica of node4's primary module, module4 (*Setting*) will be activated and served from node2.

After the selection process has finished as in figure 5-9, IS commands the selected neighbor, node2 to assume the primary task of node4. The network card on node2 will create a virtual IP for node4. Comparison that depicts this changes can be seen in figure 5-10 and figure 5-11. After becoming fail-over, node2 has two IP addresses on the same network interface.

```
root@node2:~
File  Edit  View  Terminal  Tabs  Help

[root@node2 ~]# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0C:29:23:71:3D
          inet addr:10.0.0.2  Bcast:10.0.0.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe23:713d/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1076 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3567 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:99232 (96.9 KiB)  TX bytes:153878 (150.2 KiB)
          Interrupt:67 Base address:0x2024

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:11810 errors:0 dropped:0 overruns:0 frame:0
          TX packets:11810 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:20289371 (19.3 MiB)  TX bytes:20289371 (19.3 MiB)
```

Figure 5.10: Before node2 becoming a fail-over

```
root@node2:~
File  Edit  View  Terminal  Tabs  Help
[root@node2 ~]# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0C:29:23:71:3D
          inet addr:10.0.0.2  Bcast:10.0.0.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe23:713d/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1384 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3921 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:127568 (124.5 KiB)  TX bytes:169382 (165.4 KiB)
          Interrupt:67 Base address:0x2024

eth0:1    Link encap:Ethernet  HWaddr 00:0C:29:23:71:3D
          inet addr:10.0.0.4  Bcast:10.0.0.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          Interrupt:67 Base address:0x2024

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:12042 errors:0 dropped:0 overruns:0 frame:0
          TX packets:12042 errors:0 dropped:0 overruns:0 carrier:0
```

Figure 5.11: Node2 becoming a fail-over

The physical effect is undertaken at proxy server. Initially the directive alias for module 4,(*Setting*) is reversed to IP address 10.0.0.4 at node4, but after recovery it is reversed to IP address 10.0.0.4 at node2 effectively activating the replica.

As a result, it is still possible to open module4 even though node4 is shut down completely. The downtime taken for recovery is insignificant and can be disregarded in terms of user experience.

As a footnote, $Rtime$ is measured starting at same point in the program using monotonic clock. $Exptime$ is calculated from the sampling of $Rtime$. In the monitoring terminal, it can be seen that there is slight discrepancies where some $Rtime$ value is slightly larger than the previous $Exptime$ by < 0.01. This can be explained by the implementation that $Rtime$ is calculated inside the signal handler. The exact time when the receive signal is generated is inside the child process and is more accurate to determine $Rtime$ because the time delay taken for the program between signal receive and going to the signal handler cannot be determined. However it is difficult to stamp this time value inside child process because child process have independent variables from the parent. As for figures 5.8 and 5.9, it can be seen that timeout value is 0.25ms and 0.16ms larger than the expected time. The same reason holds that the timeout value is measured inside the signal handler for alarm. Alarm signal is generated by the Operating System when the timer

has elapsed and cannot be stamped from the program. The delay taken until program goes to signal handler cannot be determined because it is handled by signal libraries. Therefore the exact difference between timeout and expected cannot be calculated in the program.

5.5 Conclusion

The methodology was realized using the steps described in chapter 4. In this chapter, the results acquired are presented and discussed. For failure detection, the heartbeat transaction are consistent between node to heartbeat monitor although the heartbeat are affected by the network condition and CPU load. The fault detection technique that adapts to these changes is useful to detect failure in timely manner. Furthermore, recovery is performed promptly. The neighbor replication technique does not increase in complexity with more nodes, giving the recovery time consistent for any number of nodes. The addition of parallelness inside the NABTICS system has increased module availability. Overall system availability has increased by 70%. The steps from failure until recovery is observed on HBM terminal and node terminal. On user side, downtime is barely noticed as the service is continued by the fail-over recovery autonomously and promptly.

Chapter 6

CONCLUSIONS

6.1 Conclusion

This thesis applied a number of techniques to improve the availability of a generic web application. The aim for this technique is to improve the availability of the knowledge repository known as NABTICS. The concept of modularization is first applied to NABTICS to separate the application into different independent modules. These modules are placed on four different servers that are interconnected on a private network. A distributed environment is created in this setting where the multiple sites work together serving its primary modules for a collective system in NABTICS application. To add security and enable using single IP, the servers are encapsulated by adding a proxy server for outside interaction. The new grouping is called NABTICS cluster where having multiple resources bring more functionalities and a policy for replication. The neighbor replication technique is chosen for its supremacy against other techniques and suitability with the experiment setting and requirement. With multiple hardware, a supervisory technique needs to be added to monitor the health of the servers and to direct for recovery if a failure happens on a server. Using heartbeat interaction, the servers send heartbeat message

to a monitoring service to indicate their liveness. The monitoring service maintains a sampling for the heartbeats delivery of each node. This sampling is used to predict the next incoming heartbeat. This technique is called adaptive because it considers the current condition of the nodes and network based on the sampling. The technique is suspicious when its prediction is not met, i.e. heartbeat does not arrive in time. The technique is called affirmative because it confirms the failure is indeed a failure before declaring it. Next a recovery process takes place where the best neighbor is selected to assume the task of the failed server by taking over the floating IP as its own. The results show an improvement by 70% in overall system availability after implementing the neighbor replication technique. Furthermore, the system becomes continuous in service where a down server causes only a minimal downtime as it is quickly recovered autonomously.

Replication is a widely used technique to guarantee the availability and dependability of large scale distributed systems in the presence of faults is replication and implies the use of more services or components performing the same function. Whenever a replicated entity encounters a failure another replica is switched on and takes its place. Data and service replication is a reliability improvement technique used in many types of distributed systems. By replicating the data and services over multiple nodes, the system can support the failure of some of these nodes, without losing its ability to function

correctly.

6.2 Future Work

There are many possibilities in this framework. It depends on which aspect we would like to improve or add on.

The methodology first of all can be improved by making it more autonomous by making a daemon program that starts at boot time. Care must be taken to make sure both sides are ready for heartbeat transaction.

As mentioned in the result section, the system can withstand two consecutive failures leaving only two nodes to be fully functioning as the system is specified. At this point, two nodes must be alive to have 100% functionality of the system which is worse than the original version where one node left can perform the full function. This is because in the new version the last node is not fully replicated. A technique that can ensure the last two nodes become complete backup of each other would keep the availability up at the event of having two nodes already failed. In this context, a self run replication technique may be the answer.

The author believes that there are some aspects to a system that are not fail-stop. For example a system having a database problem is not fail-stop

but instead it goes on with a fault. While possibly a complete recovery is not necessary, it may still need to be fixed or reboot. Self reboot is too dangerous to be allowed. In the context of this thesis, a possible method to improve this is to inform the Heartbeat Monitor of a problem or request that the server has. An atomic way to do this is possibly by embedding a code message in the heartbeat. It could be as simple as a raised flag to indicate a certain problem like database crash or request to be reboot. On the server side, a local resource monitor program can be created and coupled with the heartbeat generator program. Upon receiving the request or report from the server, HBM can decide a necessary action or pass the problem to Index Server. The future works are suggested presumably.

As a summary, the thesis has proposed a technique to increase the availability of a system application by employing several existing techniques in fault detection, heartbeat monitoring and replication and presented the results that support the methodology.

REFERENCES

[1] S. Gokhale, J. Crigler, W. Farr, and D. Wallace, “System Availability Analysis Considering Hardware/Software Failure Severities,” in Software Engineering Workshop, 005. 29th Annual IEEE/NASA, pp. 47–56, April 2005.

[2] G. Shelly, Discovering Computers 2010 : Living in A Digital World : Complete. Boston, MA: Course Technology, Cengage Learning, 2010.

[3] S. Valente and A. Grimshaw, “Replicated Grid Resources,” 2011 IEEE/ACM 12th International Conference on Grid Computing, pp. 198–206, Sept. 2011.

[4] R. V. Renesse, Y. Minsky, and M. Hayden, “A Gossip-Style Failure Detection Service,” pp. 1–16.

[5] J. Gray, D. E. Corp, and D. P. Siewiorek, “High- Availability Computer Systems,” 1980.

[6] Costan, C. Dobre, F. Pop, C. Leordeanu, and V. Cristea, “A Fault Tolerance Approach for Distributed Systems Using Monitoring Based Replication,” Proceedings of the 2010 IEEE 6th International Conference on Intelligent Computer Communication and Processing, Aug. 2010.

[7] F. G. Khan, K. Qureshi, and B. Nazir, “Performance Evaluation of Fault Tolerance Techniques in Grid Computing System,” Computers & Electrical Engineering, vol. 36, pp. 1110–1122, Nov. 2010.

[8] Tchana, L. Broto, and D. Hagimont, “Approaches to Cloud Computing Fault Tolerance,” in Computer, Information and Telecommunication Systems (CITS), 2012 International Conference on, pp. 1–6, May 2012.

[9] “ANSI/IEEE: Standard Glossary of Software Engineering Terminology, std-729-1991 (1991),”

[10] Katukoori, “Standardizing Availability Definition,” University of New Orleans, New Orleans, La., USA, 1995.

[11] K. Schmidt, High Availability and Disaster Recovery Concepts, Design, Implementation. Berlin London: Springer, 2006.

[12] W. Jia, “Distributed Network Systems : From Concepts to Implementations,” 2005.

[13] J. L. Romeu, “Selected Topics in Assurance Related Technologies,” vol. 11, no. 5, 2005.

[14] Koren and C. M. Krishna, Fault Tolerant Systems.

[15] T. Ma and J. Hillston, “Evaluation of the QoS of Crash-Recovery Failure Detection Categories and Subject Descriptors,” pp. 538–542.

[16] S. S. Sathya and K. S. Babu, “Survey of Fault Tolerant Techniques for Grid,” Computer Science Review, vol. 4, no. 2, pp. 101 – 120, 2010.

[17] T. D. Chandra and S. Toueg, “Unreliable Failure Detectors for Reliable Distributed Systems,” J. ACM, vol. 43, pp. 225–267, Mar. 1996.

[18] L. Falai and a. Bondavalli, “Experimental Evaluation of the QoS of Failure Detectors on Wide Area Network,” 2005 International Conference on Dependable Systems and Networks (DSN’05), pp. 624–633, 2005.

[19] K. Matsudaira, “Scalable Web Architecture and Distributed Systems,” The Performance of Open Source Applications : Speed Precision, and a Bit of Serendipity, 2013.

[20] A. S. M. Noor, “Data Neighbor Replica Affirmative Adaptive Failure Detection and Autonomous Recovery,” Dissertation for Doctor of Philosophy in Computer Science, 2012.

[21] S. Song, “Survey on Scalable Failure Detectors,”

[22] R. Subramaniyan, P. Raman, A. D. George, and M. Radlinski, “GEMS: Gossip-Enabled Monitoring Service for Scalable Heterogeneous Distributed Systems,” Cluster Computing, vol. 9, pp. 101–120, Jan. 2006.

[23] R. Mamat, M. M. Deris, and M. Jalil, “Neighbor Replica Distribution Technique for Cluster Server Systems,” Malaysian Journal of Computer Science, vol. 17, pp. 11–20, 2004.

[24] T. Amjad, M. Sher, and A. Daud, “A Survey of Dynamic Replication Strategies for Improving Data Availability in Data Grids,” Future Generation Computer Systems, vol. 28, pp. 337–349, Feb. 2012.

[25] D. Ford, F. I. Popovici, M. Stokely, V.-a. Truong, L. Barroso, C. Grimes, and S. Quinlan, “Availability in Globally Distributed Storage Systems,”

[26] H. Lin, K. Chen, and X. Yan, Astrolabe: a Grid Operating Environment with Full-fledged Usability. Aug 2007.

[27] F. Haas, “Ahead of the Pack: the Pacemaker High-Availability Stack,” Linux Digital Journal Magazine, pp. 98–100, April 2012.

[28] A. S. M. Noor and M. M. Deris, “Fail-stop Failure Recovery in Neighbor Replica Environment,” Procedia Computer Science, vol. 19, pp. 1040–1045, 2013.

[29] A. Zakaria,W. Awang, Z. Mohamad, A. Rose, and M. Deris, “Improving Response Time, Availability and Reliability Through Asynchronous Replication Technique in Cluster Architecture of Web Server Cluster,” in Database Theory and Application, Bio-Science and Bio-Technology

[30] (Y. Zhang, A. Cuzzocrea, J. Ma, K. I. Chung, T. Arslan, and X. Song, eds.), vol. 118 of Communications in Computer and Information Science, pp. 29–36, Springer Berlin Heidelberg, 2010.

[31] Microsoft Technet, “What is a server cluster?).” https://technet.microsoft.com/en-us/library/cc785197%28v=ws.10%29.aspx March 2003

[32] “Mysql Manual: Chapter 16 Replication,” 2015. J. Barreto, “Increasing Availability – the Reap Principles (Redundancy, Entanglement, Awareness and Persistence).”

[33] M. Blanke, M. Staroswiecki, and N. E. Wu, “Concepts and Methods in Fault-tolerant Control,” June, 2001.

[34] F. Classification, L.-b. Recovery, S. Paging, R. With, C. Transactions, B. Management, A. R. Techniques, and R. B. Systems, “Chapter 17: Recovery System”

[35] M. Clusters and L. Balancers, “Ensuring High Availability - A White Paper,”

[36] R. Huang, X. Xie, J. Chen, and Y. Lu, “Failure Detection Capability Analysis of Implementing Parallelism in Adaptive Random Testing Algorithms Categories and Subject Descriptors,” pp. 1049–1054.

[37] T. Ma, J. Hillston, and S. Anderson, “On the Quality of Service of Crash-Recovery Failure Detectors,” 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN’07), pp. 739–748, June 2007.

[38] G. Allen, D. Angulo, A. Arpaci-Dusseau, “Grid Resource Management: State of the Art and Future Trends” 2012

[39] A. Meera and S. Swamynathan, “Agent based Resource Monitoring System in IaaS Cloud Environment,” Procedia Technology, vol. 10, pp. 200–207, 2013.

[40] W. Mickens, B. D. Noble, and A. Arbor, “Exploiting Availability Prediction in Distributed Systems,” pp. 73–86.

[41] B. M. Moniruzzaman, “A Low Cost Two-Tier Architecture Model For High Availability Clusters Application Load Balancing,”

[42] A. S. M. Noor, M. M. Deris, M. Y. M. Saman, and E. A. Sirajudin, “Distributed Dynamic Failure Detection,” Journal of Software, vol. 9, pp. 1342–1347, May 2014.

[43] Shi, S. Yang, Q. Zhang, and A. F. D. Model, “Research and Analysis of Adaptive Failure Detection Algorithm,” no. August, pp. 21–24, 2010.

[44] J. Hassine, “Describing and Assessing Availability Requirements in The Early Stages of System Development,” Software & Systems Modeling, pp. 1–25, 2013.

[45] Q. Z. Sheng, X. Qiao, A. V. Vasilakos, C. Szabo, S. Bourne, and X. Xu, “Web Services Composition: A Decade’s Overview,” Information Sciences, vol. 280, pp. 218–238, Oct. 2014.

[46] J. Kangasharju, “Chapter 4 : Distributed Systems : Replication and Consistency,” 2013.

[47] Arrieta-Salinas, J. E. Armendáriz-Iñigo, J. R. Juárez-Rodríguez, and J. R. González de Mendívil, An Implementation of a Replicated File Server Supporting The Crash-Recovery Failure Model, vol. 59. Apr. 2010.

[48] Apa, O. Dieste, E. G. Espinosa G., and E. R. Fonseca C., “Effectiveness for Detecting Faults Within And Outside The Scope of Testing Techniques: An Independent Replication,” Empirical Software Engineering, vol. 19, pp. 378–417, Aug. 2013.

[49] Mansouri, G. H. Dastghaibyfard, and E. Mansouri, “Combination of Data Replication and Scheduling Algorithm for Improving Data Availability in Data Grids,” Journal of Network and Computer Applications, vol. 36, pp. 711–722, Mar. 2013.

[50] Ma, W. Liu, and T. Glatard, “A Classification of File Placement and Replication Methods on Grids,” Future Generation Computer Systems, vol. 29, pp. 1395–1406, Aug. 2013.

[51] B.-Y. Ooi, H.-Y. Chan, and Y.-N. Cheah, “Dynamic Service Placement and Replication Framework to Enhance Service Availability Using Team Formation Algorithm,” Journal of Systems and Software, vol. 85, pp. 2048–2062, Sept. 2012.

[52] A. Juan and J. M. Marquès, “Reliability and Availability Issues in Large-Scale Distributed Systems,” pp. 2915–2926, 2010.

[53] Y. Wu and G. Huang, “Model-Based High Availability Configuration Framework for Cloud,” Proceedings of the 2013 Middleware Doctoral Symposium on - MDS ’13, pp. 1–6, 2013.

[54] T. Suganuma, A. Koseki, K. Ishizaki, Y. Ueda, K. Mizuno, D. Silva, H. Komatsu, and T. Nakatani, “Distributed and Fault-Tolerant Execution Framework for Transaction Processing Categories and Subject Descriptors,” 2011

[55] Y. Kim, G. W. Bond, E. Cheung, T. M. Smith, and H. Schulzrinne, “An Evaluation Framework for Highly Available and Scalable SIP Server Clusters,” Proceedings of the

5th International Conference on Principles, Systems and Applications of IP Telecommunications - IPTcomm ’11, p. 1, 2011.

[56] U. F. Minhas, S. Rajagopalan, B. Cully, A. Aboulnaga, K. Salem, and A. Warfield, “RemusDB: Transparent High Availability for Database Systems,” The VLDB Journal, vol. 22, pp. 29–45, Oct. 2012.

[57] Undheim, A. Chilwan, and P. Heegaard, “Differentiated Availability in Cloud Computing SLAs,” 2011 IEEE/ACM 12th International Conference on Grid Computing, pp. 129–136, Sept. 2011.

[58] Wiesmann, F. Pedone, B. Kemme, and G. Alonso, “Database Replication Techniques : a Three Parameter Classification * Institute of Information Systems,” pp. 206–215, October 2000.

[59] Kanso, M. Toeroe, and F. Khendek, “Comparing Redundancy Models for High Availability Middleware,” Computing, vol. 96, pp. 975–993, Oct. 2013.

[60] Lai, M. Xie, K. Poh, Y. Dai, and P. Yang, “A Model for Availability Analysis of Distributed Software/Hardware Systems,” Information and Software Technology, vol. 44, pp. 343–350, Apr. 2002.

[61] S. Fu, “Failure-Aware Resource Management for High-Availability Computing Clusters with Distributed Virtual Machines,” Journal of Parallel and Distributed Computing, vol. 70, pp. 384–393, Apr. 2010.

[62] L.-G. Cretu, “Event-Driven Replication in Distributed Systems,” Proceedings of the 4th India Software Engineering Conference on - ISEC ’11, pp. 95–98, 2011.

[63] Turenne and A. Kanso, “A Tool Chain for Generating the Description Files,” pp. 867–870, 2014.

[64] K. Nagano, T. Itokawa, T. Kitasuka, and M. Aritsugi, “Exploitation of Backup Nodes for Reducing Recovery Cost in High Availability Stream Processing Systems,” Proceedings of the Fourteenth International Database Engineering & Applications Symposium on - IDEAS ’10, pp. 61–63, 2010.

[65] May, “Dynamic Service Quality and Resource Negotiation for High-Availability Service-Oriented Systems,” Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering – ICSE ’10, vol. 2, p. 327, 2010.

[66] J. B. Leners, H. Wu, W.-L. Hung, M. K. Aguilera, and M. Walfish, “Detecting Failures in Distributed Systems with the Falcon Spy Network,” Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles - SOSP ’11, p. 279, 2011.

[67] H. Chan and T. Chieu, “An Approach to High Availability for Cloud Servers with Snapshot Mechanism,” Proceedings of the Industrial Track of the 13th ACM/IFIP/USENIX International Middleware Conference on - MIDDLEWARE ’12, pp. 1–6, 2012.

[68] S. Rajagopalan, “Pico Replication : A High Availability Framework for Middleboxes,” October 2013.

[69] K. Ravindran, “Self-Assessment and Reconfiguration Methods for Autonomous Cloud-based Network Systems,” 2013 IEEE/ACM 17th International Symposium on Distributed Simulation and Real Time Applications, pp. 87–94, Oct. 2013.

[70] K. An, S. Shekhar, F. Caglar, A. Gokhale, and S. Sastry, “A Cloud Middleware for Assuring Performance and High Availability of Soft Real-Time Applications,” Journal of Systems Architecture, vol. 60, pp. 757–769, Oct. 2014.

[71] Cotroneo, N. Mazzocca, L. Romano, and S. Russo, “Building a Dependable System from a Legacy Application with CORBA,” Journal of Systems Architecture, vol. 48, pp. 81–98, Sept. 2002.

[72] F. Pedone and S. Frø lund, “Pronto: High availability for Standard off-the-shelf Databases,” Journal of Parallel and Distributed Computing, vol. 68, pp. 150–164, Feb. 2008.

[73] J. Hur and K. Kang, “Dependable and Secure Computing in Medical Information Systems,” Computer Communications, vol. 36, pp. 20–28, Dec. 2012.

[74] A. Rotem-Gal-Oz, “Fallacies of Distributed Computing Explained,” http://www. rgoarchitects. com/Files/fallacies. pdf, p. 20, 2006.

[75] Koren and C. M. Krishna, “Fault-Tolerant Systems,” 2007.

[76] An, S. Shekhar, F. Caglar, A. Gokhale, and S. Sastry, “A Cloud Middleware for Assuring Performance and High Availability of Soft Real-Time Applications,” Journal of Systems Architecture, vol. 60, no. 9, pp. 757 – 769, 2014.

[77] Gray and D. P. Siewiorek, “High Availability Computer Systems,” 1991.

[78] Costan, C. Dobre, F. Pop, C. Leordeanu, and V. Cristea, “A Fault Tolerance Approach for Distributed Systems Using Monitoring Based Replication,” pp. 451–458, 2010.

[79] A. S. M. Noor and M. M. Deris, “Fail-Stop Failure Recovery in Neighbor Replica Environment,” Procedia Computer Science, vol. 19, p. 1040, 2013.

[80] May, “Dynamic Service Quality and Resource Negotiation for High-Availability Service-Oriented Systems,” Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering – Volume 2, pp. 327–328, 2010.

[81] Ford, F. Labelle, F. Popovici, M. Stokely, V.-A. Truong, L. Barroso, C. Grimes, and S. Quinlan, “Availability in Globally Distributed Storage Systems,” Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation, 2010.

[82] “ITU-T: E.800: Terms and Definitions Related to Quality of Service and Network Performance Including Dependability,” 2008.

[83] W. Minnebo and B. Van Houdt, “A Fair Comparison of Pull and Push Strategies in Large Distributed Networks,” IEEE/ACM Trans. Netw., vol. 22, pp. 996–1006, June 2014.

[84] A. Kaur and S. Verma, “Performance Measurement and Analysis of High-Availability Clusters,” SIGSOFT Softw. Eng. Notes, vol. 40, pp. 1–7, Apr. 2015.

[85] Mitchell, Advanced Linux programming. Indianapolis, Ind: New Riders Pub, 2001.

Appendix A

INDEX LOGS

Table A.1: Neighbors relationship among servers

Node's Name	First Neighbor	Second Neighbor
node1	node2	node3
node2	node1	node4
node3	node2	node4
node4	node2	node3

Table A.2: Node's Information

Node's Name	IP Address	Status	Netmask	Fail-over
node1	10.0.0.1	1	255.255.255.0	0
node2	10.0.0.2	1	255.255.255.0	0
node3	10.0.0.3	1	255.255.255.0	0
node4	10.0.0.4	1	255.255.255.0	0

Table A.3: Node's Weight to Determine Best Neighbor

Node's Name	Reliability	Weight	Volume
node1	1	2	3
node2	10	5	1
node3	2	3	1
node4	10	10	1

Appendix B

SOURCE CODES

APPENDIX B

```
/*      author : Emma Ahmad Sirajudin  */
/*      title  : hbm.c                 */
/*      description  :
        Heartbeat monitoring program for HBM
        - compile and link with rt
          $ gcc -W hbm.c -o hbm -lrt */


#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <errno.h>
#include <time.h>

/* global vars */

int conn; int count, lastcount;
int client_sock;
int timeup, nodefail, reset, nodealive, nosock;
double rtime, exptime, avgtime, oldavgtime;
char redirect[]=" >>/dev/null"; char activenode[];
char pingcmd[30]="ping -c 1 ";
char sshcmd[60]="ssh root@indexserver ./performrecovery.sh ";
int port=8888; //must be same port with heartbeat generator

struct itimerval timervalue, timervalue2;
struct sigaction timer, hbreceive, nosocket;
struct timespec emptystruct, tim2, tim3,tim4;
double t2,t1, elapsedtime,time2,time1,timertime1, elapsedtime2;
pid_t childpid, i, ppid;

/* function prototype */
void timer_handler(void);
void hb_receive(void);
void no_socket(void);

int main(int argc, char *argv[])
{

/* initializer */
      timeup = 0;
                    nodefail = 0; nodealive = 1;
      elapsedtime=0;
      oldavgtime = 8;
      exptime = 8; rtime = 0; avgtime = 0;
      conn = 0;
      strcpy(activenode,argv[1]);
      count = 0; lastcount = 0;
      int recv_exit;
      int exptime1;
      int temp2;
      double temp3;
      timervalue2.it_interval.tv_sec = 0;
      timervalue2.it_interval.tv_usec = 0;
      timervalue2.it_value.tv_sec = 0;
      timervalue2.it_value.tv_usec = 0;
      hbreceive.sa_handler = hb_receive;
      hbreceive.sa_flags = SA_NOMASK;
      timer.sa_handler = timer_handler;
      timer.sa_flags = SA_NOMASK;
```

```
        nosocket.sa_handler = no_socket;
        nosocket.sa_flags = SA_NOMASK;
        int socket_desc, c , read_size, ret, j, in=0, ans;
        struct sockaddr_in server , client;
        char client_message[4], readbuf[8];
        strcat(pingcmd,activenode);
        strcat(sshcmd,activenode);
        strcat(pingcmd,redirect);
        ppid=getpid();
        printf("Monitoring is started for %s.\n\n",activenode);
        printf("Checking :\n");
        printf("- ping cmd: %s\n",pingcmd);
        printf("- new ssh cmd: %s\n",sshcmd);
        printf("- active node: %s\n", activenode);

/* Creating socket */
        socket_desc = socket(AF_INET , SOCK_STREAM , 0);
        if (socket_desc == -1)
        {
        printf("Could not create socket.\n");
        }
        printf("Socket created.\n");

        server.sin_family = AF_INET;
        server.sin_addr.s_addr = INADDR_ANY;
        server.sin_port = htons(port);

        if( bind(socket_desc,(struct sockaddr *)&server , sizeof(server)) < 0)
        {
        perror("bind failed. Error");
        return 1;
        }
        printf("bind done.\n");
              listen(socket_desc , 3);

        printf("Waiting for incoming connections....\n");
        c = sizeof(struct sockaddr_in);

            while(count<50)
                    {
                            printf("\n\n%d. ",count);
                            if(((timeup==1) || (nosock>2)) && (nodefail==0))  //timeup has happened previously
                            {
                                    printf("ping and waiting for response\n");
                                    if ((system(pingcmd))==0)
                                    {
                                            nodefail=0;
                                            printf("\n=== Ack ping reply %s ===    [OK]\n", activenode);
                                            timeup=0;
                                            exptime=8;
                                            oldavgtime=8;
                                            reset=1;
                                            nosock=0;
                                    }
                                    else
                                    {
                                            nodefail=1;
                                            printf("\n=== %s current status        [FAILED]\n", activenode);
                                            printf("Will perform recovery on %s\n", activenode);
                                            printf("%s\n",sshcmd);
                                            if ((system(sshcmd))==0)
                                            {
                                                    printf("success\n");
                                                    timeup=0;
                                                    nodefail=0;
                                                    exptime=8;
```

```
                                        oldavgtime=8;
                                        reset=1;
                                        nosock=0;
                                }
                                else printf("error\n");
                        }
                }
                if ((timeup==0) && (nosock<=3)){

                /* processing expected time to set timer*/
                temp3 = exptime*1000;
                temp2 = (int)temp3;
                exptime1 = temp2;
                rtime = 0;
                timervalue.it_interval.tv_sec = exptime1 / 1000;
                timervalue.it_interval.tv_usec = (exptime1 % 1000) * 1000;
                timervalue.it_value.tv_sec = exptime1 / 1000;
                timervalue.it_value.tv_usec = (exptime1 % 1000) * 1000;
                printf("Timer set\n");

        /* Timer 1: */
                clock_gettime(CLOCK_MONOTONIC,&tim2);
                setitimer(ITIMER_REAL, &timervalue, NULL);
                time2=tim2.tv_sec+(tim2.tv_nsec/10000000000.0);
                sigaction(SIGALRM, &timer, NULL );   // timer will stop either by SIGALRM
                sigaction(SIGUSR1, &hbreceive, NULL ); // or SIGUSR1
                sigaction(SIGUSR2, &nosocket, NULL ); // when socket cant be created

        /* Starting child process */
                childpid=fork();
                  if (childpid >= (pid_t) 0) /* fork succeeded */
                  {
                        if (childpid > 0)   /*parent*/
                        {
                           i = childpid;
                        }
                        else if (childpid == 0) /* child */
                        {
                                client_sock=accept(socket_desc, (struct sockaddr *)&client,
(socklen_t*)&c);
                                if (client_sock <= 0)
                                {
                                        printf("\n**DANGER: socket not created. %d
times\n",nosock+1);

                                        kill(ppid, SIGUSR2);   //send signal to Parent
                                        exit(0);
                                        break;
                                }


                                while((recv_exit=recv(client_sock,client_message, 5,0))<=0) {}
//just wait for heartbeat

                                        if((strcmp(client_message,activenode))!=0)
                                        {
                                                printf("DANGER: %s is not the right heartbeat
message for %s.\n",client_message,activenode);
                                                strcpy(client_message,"xxxxx");
                                                close(client_sock);
                                                kill(getpid(),SIGTERM); //kill self
                                                exit(0);
                                                break;
                                        }
                                        else
                                        {
                                                strcpy(client_message,"xxxxx");
```

```
                                                                        close(client_sock);
                                                                        kill(ppid, SIGUSR1);   //send signal to Parent
                                                                        exit(0);
                                                                        break;
                                                                }
                                                        }
                                                }
                                        }
                                sleep(2); //may change according to heartbeat generator frequency but this cycle must be slower
                                count++;
                } //end of while

                close(client_sock);
                return 0;
} //end of function main

void timer_handler()
{
                timeup=1;
                setitimer(ITIMER_REAL, &timervalue2, NULL);
                clock_gettime(CLOCK_MONOTONIC,&tim3); kill(i,SIGTERM);
                timertime1=tim3.tv_sec+(tim3.tv_nsec/10000000000.0);
                elapsedtime2=timertime1-time2;
                printf("\n*****%s TIMEOUT while exptime %f ms\n",activenode,exptime);
                return;
}

void hb_receive()
{
                clock_gettime(CLOCK_MONOTONIC,&tim4);
                setitimer(ITIMER_REAL, &timervalue2, NULL);
                time1=tim4.tv_sec+(tim4.tv_nsec/1000000000.0);
                elapsedtime=time1-time2;
                printf("Ack Hbeat %s Rtime=%f ms, Exptime=%f
                ms\n==============",activenode,elapsedtime,exptime);
                double temp = oldavgtime + elapsedtime;
                exptime = temp*0.5;
                oldavgtime = exptime;
                return;
}

void no_socket()
{
                int temp = count-lastcount;
                if(temp==1) //consecutive three times of no socket created
                {
                                nosock++;
                }
                else nosock=1;
                lastcount=count;
                return;
}


//end of file
```

```
/*      author : Emma Ahmad Sirajudin     */
/*      title  : hbgen.c                  */
/*      description  :
        Heartbeat generator program installed at node     */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <unistd.h>
#include <string.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <time.h>
int port = 8888;
struct timespec monotime1,monotime2;  double time2,time1, elapsedtime;

int main(int argc , char *argv[])
{
   int sock, i=0;
   clock_gettime(CLOCK_MONOTONIC,&monotime1);
   time2=monotime1.tv_sec+(monotime1.tv_nsec/10000000000.0); time1=0; elapsedtime=0;
   struct sockaddr_in server;
   const char msg[]="node4\0";  //put the right message i.e. node name

   server.sin_addr.s_addr = inet_addr("10.0.0.10");
   server.sin_family = AF_INET;
   server.sin_port = htons(port);

   puts("Connected\n");

   while(i<50)
   {
                clock_gettime(CLOCK_MONOTONIC,&monotime1);
                time2=monotime1.tv_sec+(monotime1.tv_nsec/10000000000.0);
                sock = socket(AF_INET , SOCK_STREAM , 0);
                if (sock == -1)
                {
                        printf("Could not create socket");
                }
                int x = connect(sock , (struct sockaddr *)&server , sizeof(server));
                if (x < 0)
                {
                        perror("connect failed. Error");
                        return 1;
                }
                if( send(sock,msg,strlen(msg), 0) < 0)
                {
                        printf("Send failed %d",i);
                        perror("error");
                        printf("\n %d",errno);
                }
                clock_gettime(CLOCK_MONOTONIC,&monotime2);
                time1=monotime2.tv_sec+(monotime2.tv_nsec/1000000000.0);
                elapsedtime=time1-time2;
                printf("%f\n",elapsedtime);//testing purpose
                sleep(1);
                close(sock);
                i++;
   }//end of while

   return 0;
}//end of function main

//end of file
```

```
# !/bin/sh
# Author: Emma Ahmad Sirajudin
# Title: performrecovery.sh
# Description: Select a fail-over replica for failed node and send SSH command to it to activate

shopt  -s -o nounset

declare InActiveServer=$1
START_SCRIPT=$(date +%s.%N)

#Function to perform arithmetic operations for float/real number.....
function  float_arithmetic(){
    echo | awk 'END { print  '"$1"'; }'
}

#Function to perform comparison between 2 value of float/real number.....
function  float_comparison(){
    echo | awk 'END { exit ( !( '"$1"') ); }'
}

#Find neighbors
IFS=,
 echo "   "
   echo "IndexServer:  Finding the neighbors of $InActiveServer"
   echo "   "
     nCtr=0;
            maxWeighted=0;
    while read  SvrName  neighbor1  neighbor2  neighbor3
       do
              nCtr=$(($nCtr + 1))
                if [ "$SvrName" = "$InActiveServer"  ]
                then
                                                          break
                fi


       done  <~/neighbors.txt

#Calculate weight of the neighbors
IFS=,
   echo "IndexServer:  Determining the best neighbor"
   echo "   "
   while read  Server  Rely  Bw  Vlm
       do
              nCtr=$(($nCtr + 1))
                if [ "$Server" = "$neighbor1" -o  "$Server" = "$neighbor2" -o "$Server" = "$neighbor3" ]
                then
                       #Calculate the total weighted...
                        tempWeighted=`float_arithmetic "$Rely + $Bw + $Vlm"`

                       #Perform comparison...
                        float_comparison "$tempWeighted > $maxWeighted" &&  maxWeighted=$tempWeighted  &&
mostReliableServer=$Server

                       # echo "Server=$Server   Rely=$Rely   Bw=$Bw  Vlm=$Vlm  totalWeighted=$tempWeighted"
                fi


       done  <~/weights.txt

END_SCRIPT=$(date +%s.%N)
     echo "IndexServer:  Most reliable server is $mostReliableServer for weighted = $maxWeighted"
printf "IndexServer:  Time taken to find the most reliable node is %.3F seconds\n" $(echo "$END_SCRIPT -
$START_SCRIPT"|bc)
echo " "
```

APPENDIX B

```
#UPDATE STATUS......
IFS=,
   echo "IndexServer:  Updating server status for $InActiveServer"
   echo "   "

    while read Server  ip_add  stat nmask  vir_ip  vir_data
       do
                nCtr=$(($nCtr + 1))
                 if [ "$Server" = "$InActiveServer"  ]
                 then
                        stat=0
                        vir_ip=$mostReliableServer
InActiveServerIP=$ip_add
InActiveServerNetmask=$nmask
 echo "IndexServer:  Activating the $mostReliableServer to be fail-over for $InActiveServerIP.."
echo ""
ssh root@$mostReliableServer ifconfig eth0:1 $InActiveServerIP netmask $InActiveServerNetmask up


                 fi

        done  <~/site_status.txt
        sleep .2
```

BIODATA

The author was born on 30th September 1985 in Kuala Terengganu. She attended SK and SM Padang Midin in Kuala Terengganu for her primary and secondary education. She received her Bachelor in Engineering from Vanderbilt University in Computer Engineering. The author used to work as a software engineer at Sony EMCS Sdn. Bhd. prior to serving at Top IT Industries Sdn. Bhd. as a marketing executive. Her research interests include Embedded Programming and Distributed Computing. She lives in Kuala Terengganu with her husband and son.